Robotics: Science and Systems 2026
Sydney, Australia, July 13–17, 2026

$\pi^*_{0.6}$: a VLA That Learns From Experience

Physical Intelligence
Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alexander Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, Zhiyuan Zhou
https://pi.website/blog/pistar06

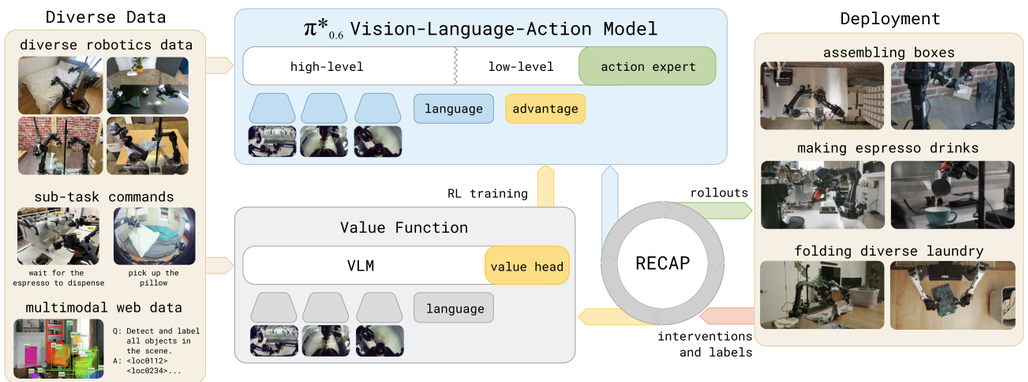


Fig. 1: **RECAP enables VLAs to improve performance during deployment.** Our model incorporates *advantage conditioning*, allowing it to ingest diverse types of real-world experience: human demonstrations, autonomous rollouts and online corrections. Learning from real-world interaction via advantage conditioning improves the model's understanding of how actions influence performance, leading to a faster and more successful policy.

***Abstract*—Vision–language–action (VLA) models offer a promising path toward general-purpose robots, but achieving the reliability and speed required for practical deployment remains challenging. We present a general-purpose method, RL with Experience and Corrections via Advantage-conditioned Policies (RECAP) that improves the efficiency and reliability of VLA policies by utilizing their real-world experience. Our method introduces value-based advantage conditioning during both pre-training and post-training phases, enabling VLA policies to ingest highly heterogeneous real-world experience, including human demonstrations, policy rollouts, and online correction data. We show that the $\pi^*_{0.6}$ model, trained with RECAP, achieves hours-long deployment of folding diverse laundry in real homes, reliably assembles boxes in a factory, and makes espresso drinks using a professional espresso machine. On some of the hardest tasks, RECAP more than doubles task throughput and roughly halves the task failure rate.**

I. Introduction

Practice makes perfect: while people are remarkably flexible in acquiring new skills, mastery invariably requires refinement through trial and error. General-purpose robotic foundation models, such as vision-language-action (VLA) models, now allow tasks to be specified flexibly via prompts. But just like people, they must *practice* to achieve mastery. This entails learning not only from demonstrations, but from autonomously collected experiential data that allows the policy to correct mistakes it makes during deployments, improve speed and robustness beyond human teleoperation, and adapt to new deployment conditions. Although the principles of learning through autonomous practice have been known for decades, as formalized with reinforcement learning (RL) [1], realizing them in a general and scalable robotic system remains challenging due to issues of scalability, data heterogeneity, and noisy reward signals.

We introduce RECAP, a training recipe that enables general-purpose vision–language–action (VLA) models to improve based on their own experiences and expert interventions. RECAP first pre-trains a VLA model on diverse, multi-task, multi-robot data via an offline RL step. During subsequent deployment, the VLA model is iteratively improved with policy rollouts, expert interventions, and sparse, binary reward feedback that reflects task success. Our recipe allows the model not only to imitate demonstrations, but to adapt to real deployment conditions, correct its own mistakes and improve efficiency beyond human teleoperation.

Training with RECAP follows an offline RL [2, 3] recipe (Figure 1): We first train a value function that estimates progress toward task completion from all (multi-task) data. This value function is then used to compute *advantages* for each action chunk in the data. We then obtain a pre-trained model that conditions the policy on a binarized improvement indicator based on this advantage [4]. After pre-training, RECAP fine-tunes the model on each downstream task with task-specific data, and then performs one or more rounds of

机器人学：科学与系统 2026，澳大利
亚悉尼，2026年7月13–17日

$\pi^*_{0.6}$：一种从经验中学习的 VLA

Physical Intelligence
Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, VishnuMano, Mohith Mothukuri, SurajNair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alexander Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, Zhiyuan Zhou
https://pi.website/blog/pistar06

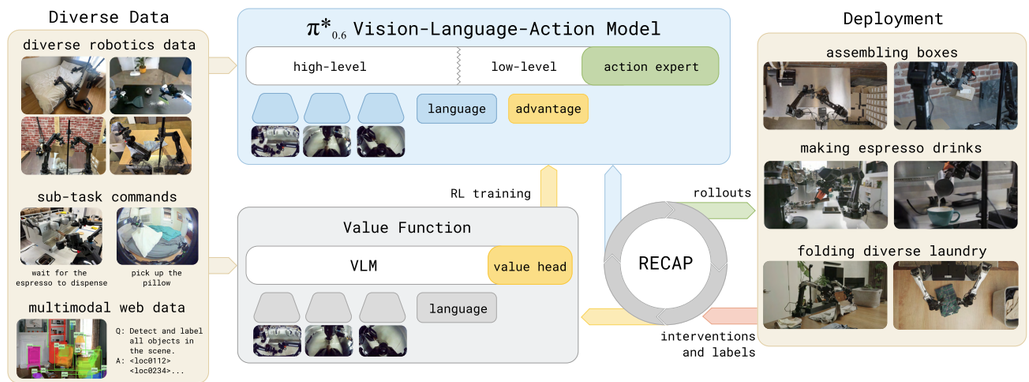


图 1：RECAP 使 VLA 能够在部署期间提升性能。我们的模型引入了优势条件化，使其能够吸收多种类型的真实世界经验：人类示范、自主策略展开和在线纠正。通过优势条件化从真实世界交互中学习，可提升模型对动作如何影响性能的理解，从而带来更快、更成功的策略。

摘要——视觉—语言—动作（VLA）模型为实现通用机器人提供了一条有前景的路径，但要达到实际部署所需的可靠性和速度仍然具有挑战性。我们提出一种通用方法——通过优势条件化策略进行经验与纠正强化学习（RL with Experience and Corrections via Advantage-conditioned Policies, RECAP），它利用 VLA 策略的真实世界经验来提升其效率和可靠性。我们的方法在预训练和后训练阶段均引入基于价值的优势条件化，使 VLA 策略能够吸收高度异构的真实世界经验，包括人类示范、策略展开和在线纠正数据。我们表明，经 RECAP 训练的 $\pi^*_{0.6}$ 模型能够在真实家庭中连续数小时部署折叠各类衣物，在工厂中可靠地组装箱子，并使用专业意式浓缩咖啡机制作浓缩咖啡饮品。在一些最难的任务上，RECAP 将任务吞吐量提高一倍以上，并将任务失败率约降低一半。

I. 引言

熟能生巧：尽管人们在新技能的习得方面表现出非凡的灵活性，但要达到精通，始终需要通过试错来不断完善。通用机器人基础模型（例如视觉-语言-动作（VLA）模型）如今使任务能够通过提示被灵活指定。但就像人一样，它们必须通过练习才能达到精通。这意味着它们不仅要从示范中学习，还要从自主收集的经验数据中学习；这些数据使策略能够纠正在部署期间所犯的错误、将速度和鲁棒性提升到超出人类遥操作的水平，并适应新的部署条件。尽管通过自主练习来学习的原则数十年来已为人所知，并已由强化学习（RL）[1], 加以形式化，但要在通用且可扩展的机器人系统中实现这些原则仍然具有挑战性，原因在于可扩展性、数据异质性和带噪奖励信号等问题。

我们提出 RECAP，一种训练方案，使通用视觉—语言—动作（VLA）模型能够基于自身经验与专家干预进行改进。RECAP 首先通过离线强化学习步骤，在多样化的多任务、多机器人数据上预训练 VLA 模型。在后续部署过程中，VLA 模型通过策略 rollout、专家干预以及反映任务成功的稀疏二值奖励反馈进行迭代改进。我们的方案使模型不仅能够模仿演示，还能适应真实部署条件、纠正自身错误，并提升超越人类遥操作的效率。

使用 RECAP 进行训练遵循一个离线 RL [2, 3] 配方（图 1）：我们首先训练一个价值函数，用所有（多任务）数据估计任务完成进度。随后使用该价值函数为数据中的每个动作块计算优势。然后，我们得到一个预训练模型，该模型使策略以基于该优势的二值化改进指标为条件 [4]。在预训练之后，RECAP 使用任务特定数据在每个下游任务上微调模型，然后执行一轮或多轮

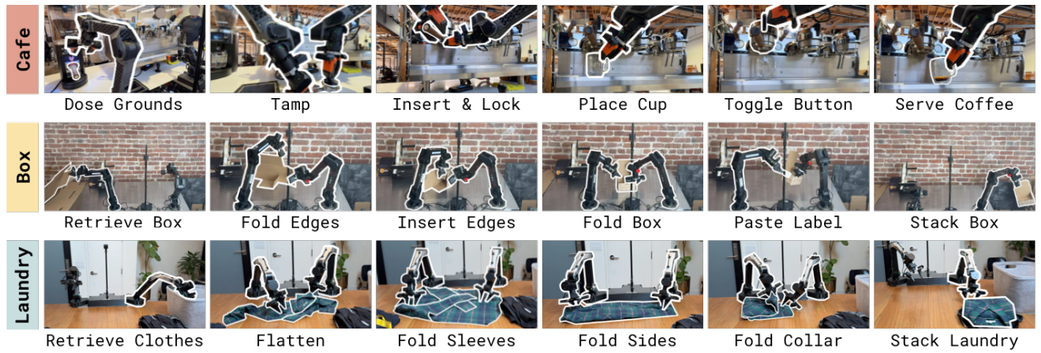


Fig. 2: **Real-world tasks improved by RECAP.** We evaluate RECAP on long-horizon manipulation tasks that require dexterous, multi-stage behavior: making espresso drinks, assembling cardboard boxes, and folding diverse laundry. These tasks include realistic sources of variability, such as deformable clothing, flattened boxes that stick together or bend, and liquid handling during espresso preparation.

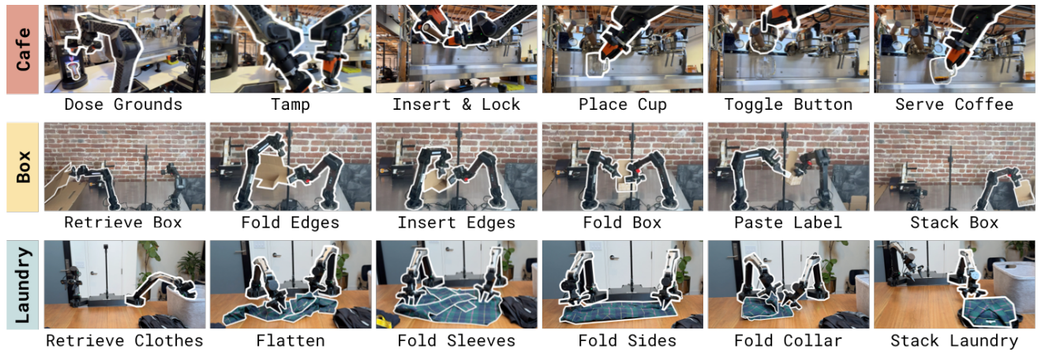


图 2：RECAP 改进的真实世界任务。我们在需要灵巧、多阶段行为的长时程操作任务上评估 RECAP：制作浓缩咖啡饮品、组装纸箱，以及折叠各式衣物。这些任务包含真实世界中的变异性来源，例如可变形衣物、会粘连或弯曲的压扁纸箱，以及制作浓缩咖啡过程中的液体处理。

on-robot data collection and offline RL to iteratively improve the policy.

We applied RECAP to challenging, long-horizon manipulation tasks such as folding diverse laundry, assembling boxes, and making espresso drinks (see Figure 2). Training with RECAP more than doubles the throughput on some of the hardest tasks, and can reduce failure rates by 2× or more, enabling levels of robustness required for real-world deployment: it ran continuously for 13 hours making espresso drinks, folded previously unseen laundry items in a new home for over two hours without interruption, and assembled boxes used in real factory packaging. While RECAP builds on algorithmic components explored in prior work, our system is novel in combining these elements, and the results show, for the first time, that a general-purpose RL recipe—combining autonomous deployment, sparse human reward feedback and expert interventions—can significantly improve both the robustness and throughput of VLA models at scale.

II. Related Work

Policies trained with imitation learning (IL) are known to suffer from compounding errors [5] and, at best, can only be as performant as the demonstration data. The goal of this work is to improve the reliability and speed of VLA policies by going beyond IL from offline demonstrations. Prior works have used online interventions to improve robotic manipulation policies [6–9]. We adopt a form of such interventions, called human-gated DAgger [8, 10]. In contrast to these works, our method uses both expert interventions and fully autonomous experience, resulting in an RL-based framework that integrates multiple data sources. There is a large body of work on using RL for autonomous improvement of robotic manipulation policies [11–19], including methods using diffusion-based policies [20–22], in multi-task settings [23, 24], and using pre-trained multi-task policies [25–27]. Unlike these works, we study how to scale real-world RL to large VLA policies for long-horizon, fine-grained manipulation tasks.

Many recent works have studied how to improve a base VLA model through RL. Several works directly apply the proximal policy optimization (PPO) algorithm and variations thereof to VLA fine-tuning [28–32], yielding approaches that are difficult to extend to real-world RL in an efficient and scalable fashion. Others have explored RL fine-tuning *on top of* pre-trained VLA models, either training a residual policy [33, 34], fine-tuning an action head network [35], selecting or refining actions proposed by the VLA [36–38], or optimizing the initial noise of a diffusion-based VLA [39]. Some have also explored ways to distill the learned behavior back into the VLA [33, 34, 36, 40]. These prior works generally use discrete actions or simple Gaussian continuous action distributions. A critical distinction is that we train an entire VLA end-to-end using iterated offline RL, with an expressive flow matching VLA model. This is made possible by a simple and scalable advantage-conditioned policy extraction method, which removes much of the complexity of using policy gradient style objectives with large VLA models. Our comparisons show that this significantly outperforms a more traditional policy gradient based extraction scheme.

More closely related to RECAP in terms of methodology, a number of prior works have integrated value functions and end-to-end RL training of VLAs on real robots [41–44]. For example, Huang et al. [41] apply calibrated Q-learning to an offline demonstration dataset for grasping tasks, without an online improvement phase. Zhang et al. [42] use direct preference optimization (DPO) to optimize pick-and-place skills from human preferences, using online rollouts from a VLA. Finally, Zhai et al. [43], Ghasemipour et al. [44] use PPO and REINFORCE, respectively, with time-to-completion value functions to train VLAs for tasks like moving a bowl, unfolding a mat, and pushing objects on a table. In contrast to these prior works, we describe an iterated offline RL framework for VLAs with multiple advantages. First, our method supports high-capacity diffusion and flow-based VLAs, unlike the discrete-action models studied in

机器人实机数据收集和离线 RL，以迭代改进策略。

我们将 RECAP 应用于具有挑战性的长时程操作任务，例如折叠各式衣物、组装纸箱和制作浓缩咖啡饮品（见图 2）。使用 RECAP 训练后，一些最难任务的吞吐量提升了一倍以上，并且可将失败率降低 2× 或更多，从而实现真实世界部署所需的稳健性水平：它连续运行 13 小时制作浓缩咖啡饮品，在一个新家中不间断地折叠此前未见过的衣物超过两小时，并组装了用于真实工厂包装的纸箱。虽然 RECAP 建立在先前工作中探索过的算法组件之上，但我们的系统在将这些要素结合起来方面具有新颖性，而且结果表明，一种通用 RL 方案——结合自主部署、稀疏人类奖励反馈和专家干预——首次能够显著提升大规模 VLA 模型的稳健性和吞吐量。

II. 相关工作

已知通过模仿学习（IL）训练的策略会遭受复合误差 [5]，并且最多只能达到与演示数据相同的性能。本工作旨在超越基于离线演示的 IL，提升 VLA 策略的可靠性和速度。先前工作使用在线干预来改进机器人操作策略 [6–9]。我们采用此类干预的一种形式，称为人类门控的 DAgger [8, 10]。与这些工作不同，我们的方法同时使用专家干预和完全自主经验，形成了一个集成多种数据源的基于 RL 的框架。已有大量工作研究利用 RL 自主改进机器人操作策略 [11–19]，包括在多任务设置 [23, 24]，中使用基于扩散的策略 [20–22]，的方法，以及使用预训练多任务策略 [25–27] 的方法。与这些工作不同，我们研究如何将真实世界 RL 扩展到大型 VLA 策略，以完成长时程、细粒度操作任务。

许多近期工作研究了如何通过 RL 改进基础 VLA 模型。若干工作直接将近端策略优化（PPO）算法及其变体应用于 VLA 微调 [28–32],，所得方法难以以高效且可扩展的方式扩展到真实世界 RL。其他工作探索了在预训练 VLA 模型之上进行 RL 微调，方法包括训练残差策略 [33, 34]、、微调动作头网络 [35]、、选择或精炼由 VLA 提出的动作 [36–38],，或优化基于扩散的 VLA 的初始噪声 [39]。还有一些工作探索了将学习到的行为蒸馏回 VLA 的方法 [33, 34, 36, 40]。这些先前工作通常使用离散动作或简单的高斯连续动作分布。一个关键区别在于，我们使用迭代式离线 RL，并借助具有强表达能力的流匹配 VLA 模型，对整个 VLA 进行端到端训练。这得益于一种简单且可扩展的优势条件化策略提取方法，它消除了将策略梯度式目标用于大型 VLA 模型时的许多复杂性。我们的比较表明，该方法显著优于更传统的基于策略梯度的提取方案。

在方法论上与 RECAP 更密切相关的是，许多先前工作已将价值函数与在真实机器人上对 VLA 进行端到端 RL 训练相结合 [41–44]。例如，Huang 等人 [41] 将校准的 Q-learning 应用于抓取任务的离线示范数据集，且没有在线改进阶段。Zhang 等人 [42]使用直接偏好优化（DPO），借助来自 VLA 的在线 rollout，根据人类偏好优化拾放技能。最后，Zhai 等人 [43], 和 Ghasemipour 等人 [44] 分别使用 PPO 和 REINFORCE，并结合完成时间价值函数来训练 VLA，以完成移动碗、展开垫子以及推动桌上物体等任务。与这些先前工作不同，我们介绍了一种用于 VLA 的迭代式离线 RL 框架，它具有多项优势。首先，我们的方法支持高容量扩散模型和基于流的 VLA，不同于在……中研究的离散动作模型

prior works. Second, we avoid the need for on-policy PPO or REINFORCE by using an advantage conditioning strategy for policy extraction, which can utilize all prior (off-policy or offline) data. Lastly, our evaluation consists of complex, dexterous, and temporally extended tasks, where our method increases throughput by about 2× while handling deformable objects, liquids, and multi-stage tasks.

Prior works have explored the idea of conditioning the policy on rewards, values, and advantages [45–54], including methods that use classifier-free guidance [4]. We extend this approach to pre-train and fine-tune a large-scale generalist VLA policy [55], incorporating a variety of data sources (including demonstrations, interventions, and autonomous policy rollouts) to learn real robotic manipulation tasks. Recent research has also studied how to effectively train multi-task, language-conditioned reward functions [56–62] and value functions [43, 63, 64]. Building on these works, we also train a language-conditioned distributional value function, which allows us to estimate state-action advantages.

III. Preliminaries

Reinforcement learning. We consider the standard RL setting in which an agent, given by a policy $\pi(\mathbf{a}_t|\mathbf{o}_t)$, outputs actions $\mathbf{a}_t$ given an observation $\mathbf{o}_t \in \mathcal{O}$. We define a trajectory as $\tau = (\mathbf{o}_0, \mathbf{a}_0, \cdots, \mathbf{o}_T) \in \mathcal{O} \times \mathcal{A} \cdots \mathcal{O}$. A distribution over trajectories $\rho_\pi(\tau)$ is induced by the policy $\pi(\mathbf{a}_t|\mathbf{o}_t)$ and the stochastic dynamics $p(\mathbf{o}_{t+1}|\mathbf{o}_t, \mathbf{a}_t)$: $\rho_\pi(\tau) = p(\mathbf{o}_0)\prod_{t=0}^{T-1}\pi(\mathbf{a}_t|\mathbf{o}_t)p(\mathbf{o}_{t+1}|\mathbf{o}_t, \mathbf{a}_t)$. The reward function is given by $r(\mathbf{o}_t, \mathbf{a}_t)$, and we abbreviate it to r_t to shorten notation, where r_T is the terminal reward. The goal of RL is to maximize the cumulative reward, learning a policy that maximizes $\mathcal{J}(\pi) = \mathbb{E}_{\tau\sim\rho_\pi}[R(\tau)] = \mathbb{E}_{\tau\sim\rho_\pi}[\sum_{t=0}^{T} r_t]$. The value function for a policy π is then defined as $V^\pi(\mathbf{o}_t) = \mathbb{E}_{\tau_{t+1:T}}[\sum_{t=t}^{T} r_t]$. We can then calculate the advantage of an action $\mathbf{a}_t$ as $A^\pi(\mathbf{o}_t, \mathbf{a}_t) = \mathbb{E}_{\rho_\pi(\tau)}[\sum_{t'=t}^{t+N-1} r_{t'} + V^\pi(\mathbf{o}_{t+N})] - V^\pi(\mathbf{o}_t)$, corresponding to an n-step estimate.

Regularized reinforcement learning. Instead of maximizing $\mathcal{J}(\pi)$, it is common to use regularization in RL, optimizing for a policy that maximizes reward while remaining close to some reference policy π_{ref} [65–69]. This is important, for example, when we want to train for many gradient steps on the same data, in which case π_{ref} typically corresponds to the behavior policy that collected the training data. This can be formalized via the objective $\mathcal{J}(\pi, \pi_{\text{ref}}) = \mathbb{E}_{\tau\sim\rho_{\pi_\theta}}[\sum_{t=0}^{T}\gamma^t r_t] - \beta\mathbb{E}_{\mathbf{o}\sim\rho_{\pi_\theta}}[D(\pi(\cdot|\mathbf{o})\|\pi_{\text{ref}}(\cdot|\mathbf{o}))]$, where D denotes some divergence metric. For the case where D is the KL divergence, we have the well-known result that $\hat{\pi}(\mathbf{a}|\mathbf{o}) \propto \pi_{\text{ref}}(\mathbf{a}|\mathbf{o})\exp(A^{\pi_{\text{ref}}}(\mathbf{o}, \mathbf{a})/\beta)$ is the solution to $\max_\pi J(\pi, \pi_{\text{ref}})$, with Lagrange multiplier β [66–69]. Our advantage-conditioned policy extraction method is based on a closely related but less well-known result: if we define the policy $\hat{\pi}(\mathbf{a}|\mathbf{o}) \propto \pi_{\text{ref}}(\mathbf{a}|\mathbf{o})p(I|A^{\pi_{\text{ref}}}(\mathbf{o}, \mathbf{a}))^\beta$, where $p(I|A^{\pi_{\text{ref}}}(\mathbf{o}, \mathbf{a})) = g(A^{\pi_{\text{ref}}}(\mathbf{o}, \mathbf{a}))/\int g(A^{\pi_{\text{ref}}}(\mathbf{o}, \mathbf{a}'))\mathrm{d}\mathbf{a}'$ is the probability of any action $\mathbf{a}$ improving over π_{ref} as measured by a monotonically increasing function g, then $\hat{\pi}$ is guaranteed to improve over π_{ref}, i.e., $\mathcal{J}(\hat{\pi}) \geq \mathcal{J}(\pi_{\text{ref}})$ [4, 70]. We will use this property in deriving our policy extraction method in Section IV-C. Using this definition we can then obtain a parametric policy from the closed form definition of $\hat{\pi}$ by solving the following minimization problem: $\min_\theta \mathbb{E}_{s\sim\rho_{\pi_{\text{ref}}}}[KL(\hat{\pi}, \pi_\theta)]$.

Algorithm 1 Overview of RECAP

Require: multi-task demonstration dataset $\mathcal{D}_{\text{demo}}$

// Pre-train

Train V_{pre} on $\mathcal{D}_{\text{demo}}$ using Eq. 1

Train π_{pre} on $\mathcal{D}_{\text{demo}}$ using Eq. 3 and V_{pre}

// Post-train on a target task

Initialize $\mathcal{D}_\ell$ with demonstrations for target task ℓ

Train π_ℓ^0 from π_{pre} on $\mathcal{D}_\ell$ with $I_t = \text{True}$

// Deploy and iteratively improve performance

for $k = 1$ to K **do**

  Collect deployment data with π_ℓ^{k-1} and add it to $\mathcal{D}_\ell$

  Train V_ℓ^k from V_{pre} on $\mathcal{D}_\ell$ using Eq. 1

  Train π_ℓ^k from π_{pre} on $\mathcal{D}_\ell$ using Eq. 3 and V_ℓ^k

end for

IV. RL with Experience and Corrections via Advantage-Conditioned Policies (RECAP)

We present RECAP, an RL-based training framework that enables VLA models to improve during deployment (shown in Figure 1 and Alg. 1). We use RECAP to train a VLA model that we refer to as $\pi^*_{0.6}$.

A. Overview

RECAP starts with a data set of diverse human-collected *demonstrations*, and iterates through three subroutines:

1) **Value function training.** We use all of the data collected to train a large, multi-task value function, $V^{\pi_{\text{ref}}}$, which models the expected time to task completion, thereby detecting failures or suboptimalities.
2) **Advantage conditioned training.** To improve the VLA policy, we include an optimality indicator based on advantage values derived from this value function in the VLA prefix. This "advantage conditioned" recipe provides a simple and effective way to extract improved policies from deployment data.
3) **Deployment data collection.** We run the VLA on a task, labeling each episode with task outcome labels (which determine the reward), and optionally providing human interventions that demonstrate corrections for mistakes in earlier iterations.

Our pre-training phase consists of performing steps (1) and (2) above on our entire pre-training dataset, which contains tens of thousands of hours of demonstrations from numerous tasks and a variety of different robots. Then, for each task, we perform steps (1), (2), and (3) one or more times to further improve the VLA. In practice, the specialists are each fine-tuned independently from the pre-trained model. We describe

先前工作。其次，我们通过使用用于策略提取的优势条件化策略，避免了对同策略 PPO 或 REINFORCE 的需求；该策略可以利用所有先前的（离策略或离线）数据。最后，我们的评估涵盖复杂、灵巧且时间上延展的任务；在这些任务中，我们的方法在处理可变形物体、液体和多阶段任务时，将吞吐量提高约 2× 。

先前的工作探索了将策略条件化于奖励、价值和优势的思想 [45–54]，包括使用无分类器引导的方法 [4]。我们将这一方法扩展到预训练和微调大规模通用 VLA 策略 [55]，，结合多种数据源（包括演示、干预和自主策略展开）来学习真实机器人操作任务。近期研究还探讨了如何有效训练多任务、语言条件奖励函数 [56–62] 和价值函数 [43, 63, 64]。基于这些工作，我们还训练了一个语言条件分布式价值函数，使我们能够估计状态-动作优势。

III. 预备知识

强化学习。我们考虑标准强化学习设定：其中智能体由策略 $\pi(\mathrm{a}_t|\mathrm{o}_t)$ 给出，输出动作 a_t (给定观测 $\mathrm{o}_t \in \mathcal{O}$) 。我们将轨迹定义为 $\tau = (\mathrm{o}_0, \mathrm{a}0, \cdots, \mathrm{o}_T) \in \mathcal{O}\times\mathcal{A}\cdots\mathcal{O}$。轨迹上的分布 $\rho_\pi(\tau)$ 由策略 $\pi(\mathrm{a}_t|\mathrm{o}_t)$ 和随机动力学 $p(\mathrm{o}_{t+1}|\mathrm{o}_t, \mathrm{a}_t)$ 诱导得到： $\rho_\pi(\tau) = p(\mathrm{o}_0)\prod_{t=0}^{T-1}\pi(\mathrm{a}_t|\mathrm{o}_t)p(\mathrm{o}_{t+1}|\mathrm{o}_t, \mathrm{a}_t)$。奖励函数由 $r(\mathrm{o}_t, \mathrm{a}_t)$ 给出；为缩短记号，我们将其简写为 r_t ，其中 r_T 是终端奖励。强化学习的目标是最大化累积奖励，即学习一个策略以最大化 $\mathcal{J}(\pi) = \mathbb{E}_{\tau\sim\rho_\pi}[R(\tau)] = \mathbb{E}_{\tau\sim\rho_\pi}[\sum_{t=0}^{T} r_t]$。 策略 π 的价值函数则定义为 $V^\pi(\mathrm{o}_t) = \mathbb{E}_{\tau_{t+1:T}}[\sum_{t=t}^{T} r_t]$。然后，我们可以将动作 a_t 的优势计算为 $A^\pi(\mathrm{o}_t, \mathrm{a}_t) = \mathbb{E}_{\rho_\pi(\tau)}[\sum_{t'=t}^{t+N-1} r_{t'} + V^\pi(\mathrm{o}_{t+N})] - V^\pi(\mathrm{o}_t)$，这对应于一个 n 步估计。

正则化强化学习。常见做法不是最大化$\mathcal{J}(\pi)$，而是在强化学习中使用正则化，优化一个策略，使其在最大化奖励的同时保持接近某个参考策略 π_{ref} [65–69]。这一点很重要，例如，当我们希望在同一份数据上训练许多梯度步时，此时 π_{ref} 通常对应于收集该训练数据的行为策略。这可以通过目标 $\mathcal{J}(\mathbb{E}_{\tau\sim\rho_{\pi_\theta}}[\sum_{t=0}^{T}\gamma^t r_t] - \beta\mathbb{E}_{\mathbf{o}\sim\rho_{\pi_\theta}}[D\ \pi\ \|\pi_{\text{ref}}\ ],\ )$ ($(\cdot|\mathrm{o})$ $(\cdot|\mathrm{o})$) 形式化，其中 D 表示某种散度度量。对于 D 为 KL 散度的情况，有一个众所周知的结果：$\hat{}\pi(\mathrm{a}|\mathrm{o}) \propto \pi_{\text{ref}}(\mathrm{a}|\mathrm{o})\exp(A^{\pi_{\text{ref}}}(\mathrm{o},\mathrm{a})/\beta)$ 是 $\max_\pi J(\pi, \pi_{\text{ref}})$ 的解，其中拉格朗日乘子为 β [66–69]。我们的优势条件策略提取方法基于一个密切相关但不太为人所知的结果：如果我们定义策略 $\hat{}\pi(\mathrm{a}|\mathrm{o}) \propto \pi_{\text{ref}}(\mathrm{a}|\mathrm{o})p(I|A^{\pi_{\text{ref}}}(\mathrm{o},\mathrm{a}))^\beta$，其中 $p(I|A^{\pi_{\text{ref}}}(\mathrm{o}, \mathrm{a})) = g(A^{\pi_{\text{ref}}}(\mathrm{o}, \mathrm{a}))/\int g(A^{\pi_{\text{ref}}}(\mathrm{o}, \mathrm{a}'))\mathrm{d}\mathrm{a}'$ 是任意动作 a 相对于 π_{ref} 取得改进的概率（由单调递增函数 g 度量），那么 $\hat{}\pi$ 可保证

算法 1：RECAP 概述

要求：多任务演示数据集 $\mathcal{D}_{\text{demo}}$

// 预训练

使用式 1 在 $\mathcal{D}_{\text{demo}}$ 上训练 V_{pre}

使用式 3 和 V_{pre} 在 $\mathcal{D}_{\text{demo}}$ 上训练 π_{pre}

// 在目标任务上进行后训练

使用目标任务 ℓ 的演示初始化 $\mathcal{D}_\ell$

从 π_{pre} 在 $\mathcal{D}_\ell$ 上训练 π_ℓ^0，并使用 I_t =True

// 部署并迭代提升性能

对于 $k = 1$ 到 K 执行

使用 π_ℓ^{k-1} 收集部署数据，并将其添加到 $\mathcal{D}_\ell$

使用公式 1，在 $\mathcal{D}_\ell$ 上从 V_{pre} 训练 V_ℓ^k

使用式 3 和 V_ℓ^k，在 $\mathcal{D}_\ell$ 上从 π_{pre} 训练 π_ℓ^k

结束循环

相对于 π_{ref} 有所改进，即 $\mathcal{J}(\hat{}\pi) \geq \mathcal{J}(\pi_{\text{ref}})$ [4, 70]。我们将在第 IV-C 节推导策略提取方法时使用这一性质。利用这个定义，我们可以通过求解以下最小化问题，从 $\hat{}\pi$ 的闭式定义中得到一个参数化策略：$\min_\theta \mathbb{E}_{s\sim\rho_{\pi_{\text{ref}}}}[KL(\hat{}\pi, \pi_\theta)]$。

IV. 通过优势条件策略（RECAP）利用经验与纠正进行强化学习

我们提出 RECAP，这是一种基于强化学习的训练框架，可使 VLA 模型在部署期间持续改进（见图 1 和算法 1）。我们使用 RECAP 训练一个 VLA 模型，将其称为 $\pi^*_{0.6}$。

A. 概述

RECAP 从包含多样化人工采集演示的数据集开始，并依次迭代执行三个子程序：

1) 价值函数训练。我们使用所有收集到的数据来训练一个大型多任务价值函数 $V^{\pi_{\text{ref}}}$，该函数对任务完成的预期时间进行建模，从而检测失败或次优情况。2) 优势条件训练。为了改进 VLA 策略，我们在 VLA 前缀中加入基于该价值函数推导出的优势值的最优性指标。这种“优势条件化”方案提供了一种简单而有效的方式，可从部署数据中提取改进后的策略。3) 部署数据收集。我们在任务上运行 VLA，为每个回合标注任务结果标签（这些标签决定奖励），并可选地提供人工干预，以展示对早期迭代中错误的纠正。

我们的预训练阶段包括在我们整个预训练数据集上执行上述步骤（1）和（2），该数据集包含来自众多任务和各种不同机器人的数万小时演示。然后，对于每个任务，我们执行步骤（1）、（2）和（3）一次或多次，以进一步改进 VLA。在实践中，每个专家模型均从预训练模型独立进行微调。我们描述

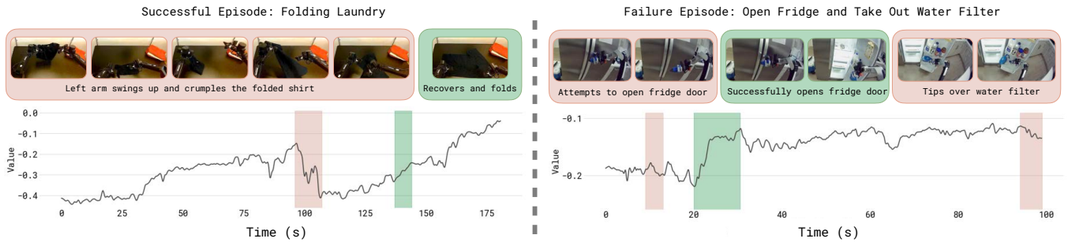


Fig. 3: **Value function visualizations.** We visualize our learned value function on a laundry folding episode that finished successfully (left), and a refrigerator manipulation episode that failed (right). The segments highlighted in red show examples of negative advantage, and green parts highlight areas of relatively high advantage, with the images on top showing the corresponding frames of the episode. These visualizations show that the value function correctly identifies mistakes in the episode as having negative advantage and models the speed of progress being made.

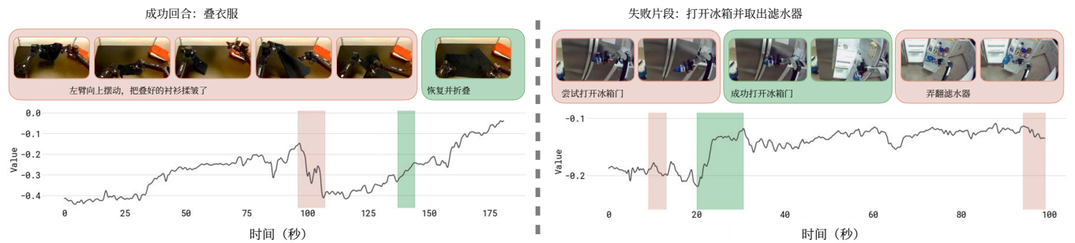


图 3：价值函数可视化。我们在一个成功完成的叠衣服片段（左）和一个失败的冰箱操作片段（右）上可视化学习到的价值函数。红色高亮片段展示了负优势的示例，绿色部分高亮了相对较高优势的区域，上方图像展示了该片段对应的帧。这些可视化表明，价值函数能够正确地将片段中的错误识别为具有负优势，并对任务推进的速度进行建模。

the value function training and policy training steps below, and then present our specific instantiation of this approach for training $\pi^*_{0.6}$ in Section V.

B. Distributional value function training

To train a value function that can act as a reliable critic for arbitrary tasks, we represent $V^{\pi_{\text{ref}}}$ with a multi-task distributional value function $p_\phi(V|\mathbf{o}_t, \ell) \in \Delta_B$ [71], mapping the observations $\mathbf{o}_t$ and language command ℓ to a distribution over B discrete bins. In our implementation, this value function uses the same architecture as the VLA policy but with a smaller VLM backbone. Using $R_t(\tau) = \sum_{t'=t}^{T} r_{t'}$ to denote the empirical return of a trajectory τ from time step t until the end, we train $p_\phi(V|\mathbf{o}_t, \ell)$ by first discretizing the empirical return value $R_t(\tau)$ into $B = 201$ bins (using R_t^B to denote the discretized returns), and then minimizing the cross-entropy H over the trajectories in the dataset $\mathcal{D}$:

$$\min_\phi \mathbb{E}_{\tau\in\mathcal{D}}\left[\sum_{\mathbf{o}_t\in\tau} H(R_t^B(\tau), p_\phi(V|\mathbf{o}_t, \ell))\right]. \quad (1)$$

This is a Monte Carlo estimator for the value function of the policy represented by the dataset $\mathcal{D}$ (i.e., the behavior policy π_{ref}). We can extract a continuous value function, and thus an advantage, from the learned value distribution using $V^{\pi_{\text{ref}}}(o_t, \ell) = \sum_{b\in[0,B]} p_\phi(V = b|\mathbf{o}_t)v(b)$, where $v(b)$ denotes the value corresponding to bin b. During the pre-training phase, the dataset $\mathcal{D}$ corresponds to the human demonstrations, and the value function captures the expected return for the task and metadata we condition on. In subsequent iterations, it skews toward a weighted combination of the return of the demonstrations and the learned policy.

While this value function can be suboptimal compared to a Q-function estimator trained via temporal difference learning, we found it to be simple and highly reliable, while still allowing for substantial improvement over imitation learning. Our method could be extended to accommodate off-policy estimators in future work.

C. Policy extraction via advantage conditioning

Once we have the value function $V^{\pi_{\text{ref}}}$, we need an effective policy extraction method that satisfies several criteria. First, it needs to be able to handle diverse off-policy data, comprising the initial demonstrations, the expert interventions, and autonomous episodes from both the latest policy and older policies. Second, it needs to be applicable to large VLA models, including models that use flow matching or diffusion to generate actions. Third, to improve the policy with deployment data, it needs to learn from both good (near-optimal) and bad (suboptimal) data.

Among the existing methods for policy extraction, policy gradient methods (including regularized and reparameterized policy gradients) are perhaps the most widely used [65, 72]. However, these methods are difficult to apply to flow matching models, which do not readily provide a tractable log-likelihood, making them hard to scale up to modern VLA architectures (see comparison in Section VI). An alternative is to use weighted regression methods, such as AWR [67, 73, 74], which implicitly provide for regularization to the behavior policy and use a simple, importance-weighted supervised learning objective. However, these methods discard or significantly downweight a sizable portion of the data, effectively implementing a kind of filtered imitation technique.

Instead, we use a variant of *advantage conditioning* [46], where the policy is trained on all of the data with an additional input indicating the *optimality* of the action. This is closely related to a variety of methods that condition the policy on some function of the resulting trajectory [45, 48]. The specific formulation in our method is most similar to CFGRL [4]. Building on the formulation in Section III, we can apply Bayes' rule to rewrite the probability of policy improvement as $p(I|A^{\pi_{\text{ref}}}(\mathbf{o}, \mathbf{a})) = \pi_{\text{ref}}(\mathbf{a}|I, \mathbf{o})/\pi_{\text{ref}}(\mathbf{a}|\mathbf{o})$. Applying this to our setting and including language conditioning, we can obtain an alternative closed form for the improved regularized policy described in Section III as

$$\hat{\pi}(\mathbf{a}, |\mathbf{o}, \ell) \propto \pi_{\text{ref}}(\mathbf{a}|\mathbf{o}, \ell)\left(\frac{\pi_{\text{ref}}(\mathbf{a}|I, \mathbf{o}, \ell)}{\pi_{\text{ref}}(\mathbf{a}|\mathbf{o}, \ell)}\right)^\beta. \quad (2)$$

For the special case $\beta = 1$, we get $\hat{\pi}(\mathbf{a}, |\mathbf{o}, \ell) = \pi_{\text{ref}}(\mathbf{a}|I, \mathbf{o}, \ell)$.

We can therefore represent $\hat{\pi}$ without needing to explicitly represent the improvement probability $p(I|A^{\pi_{\text{ref}}}(\mathbf{o}, \mathbf{a}))$, if we train the policy so that it can represent both $\pi_{\text{ref}}(\mathbf{a}|\mathbf{o}, \ell)$ and

价值函数训练和策略训练步骤将在下文描述，然后在第 V 节中介绍我们用于训练 $\pi^*_{0.6}$ 的这一方法的具体实现。

B. 分布式价值函数训练

为了训练一个能够作为任意任务可靠评价器的价值函数，我们将 $V^{\pi_{\text{ref}}}$ 表示为多任务分布式价值函数 $p_\phi(V|\mathrm{o}_t, \ell) \in !_B$ [71]，该函数将观测 o_t 和语言指令 ℓ 映射为 B 个离散桶上的分布。在我们的实现中，该价值函数使用与 VLA 策略相同的架构，但采用更小的 VLM 主干网络。使用 $R_t(\tau) = \sum_{t'=t}^{T} r_{t'}$ 表示轨迹 τ 从时间步 t 直到结束的经验回报，我们训练 $p_\phi(V|\mathrm{o}_t, \ell)$：首先将经验回报值 $R_t(\tau)$ 离散化为 $B = 201$ 个桶（用 R_t^B 表示离散化后的回报）然后最小化交叉熵 H（在数据集 $\mathcal{D}$ 的轨迹上）：

$$\min_\phi \mathbb{E}_{\tau\in\mathcal{D}}\left[\sum_{\mathbf{o}_t\in\tau} H(R_t^B(\tau), p_\phi(V|\mathbf{o}_t, \ell))\right]. \quad (1)$$

这是一个蒙特卡洛估计器，用于估计由数据集 $\mathcal{D}$（即行为策略 π_{ref}）所表示的策略的价值函数。我们可以使用 $V^{\pi_{\text{ref}}}(o_t, \ell) = \sum_{b\in[0,B]} p_\phi(V = b|\mathrm{o}_t)v(b)$ 从学习到的价值分布中提取连续的价值函数，并由此得到优势，其中 $v(b)$ 表示与分箱 b 对应的值。在预训练阶段，数据集 $\mathcal{D}$ 对应人类演示数据，价值函数刻画了在给定任务和元数据条件下的期望回报。在后续迭代中，它会偏向演示数据回报与学习到的策略回报的加权组合。

虽然与通过时序差分学习训练的 Q 函数估计器相比，该价值函数可能次优，但我们发现它简单且高度可靠，同时仍能在模仿学习的基础上实现显著提升。我们的方法在未来工作中可以扩展以支持离策略估计器。

C. 通过优势条件化提取策略

一旦我们得到价值函数 $V^{\pi_{\text{ref}}}$，就需要一种满足若干标准的有效策略提取方法。

首先，它需要能够处理多样化的离策略数据，包括初始演示、专家干预，以及来自最新策略和较旧策略的自主回合。其次，它需要适用于大型 VLA 模型，包括使用流匹配或扩散来生成动作的模型。第三，为了利用部署数据改进策略，它需要同时从良好（接近最优）和糟糕（次优）的数据中学习。

在现有的策略提取方法中，策略梯度方法（包括正则化策略梯度和重参数化策略梯度）或许是使用最广泛的方法 [65, 72]。然而，这些方法难以应用于流匹配模型，因为这类模型不易提供可处理的对数似然，从而使其难以扩展到现代 VLA 架构（参见第VI节中的比较）。另一种替代方案是使用加权回归方法，例如 AWR [67, 73, 74]，这些方法隐式地为行为策略提供正则化，并使用简单的重要性加权监督学习目标。然而，这些方法会丢弃或显著降低相当一部分数据的权重，实际上实现了一种过滤式模仿技术。

相反，我们使用优势条件化的一种变体 [46]，其中策略在所有数据上进行训练，并额外输入一个表示动作最优性的指示。这与多种方法密切相关，这些方法使策略以所得轨迹的某个函数为条件 [45, 48]。我们方法中的具体形式与 CFGRL [4] 最为相似。基于第III节中的公式，我们可以应用贝叶斯法则，将策略改进的概率重写为 $p(I|A^{\pi_{\text{ref}}}(\mathrm{o}, \mathrm{a})) = \pi_{\text{ref}}(\mathrm{a}|I, \mathrm{o})/\pi_{\text{ref}}(\mathrm{a}|\mathrm{o})$。将其应用于我们的设置并纳入语言条件化后，我们可以得到第III节中所述的改进正则化策略的另一种闭式形式：

$$\hat{\pi}(\mathbf{a}, |\mathbf{o}, \ell) \propto \pi_{\text{ref}}(\mathbf{a}|\mathbf{o}, \ell)\left(\frac{\pi_{\text{ref}}(\mathbf{a}|I, \mathbf{o}, \ell)}{\pi_{\text{ref}}(\mathbf{a}|\mathbf{o}, \ell)}\right)^\beta. \quad (2)$$

对于特殊情况 $\beta = 1$，我们得到 $\hat{\pi}(\mathrm{a}\ |\mathrm{o}\ \ell) = \pi_{\text{ref}}(\mathrm{a}|I, \mathrm{o}\ \ell)$。

因此，我们可以表示 $\hat{\pi}$，而无需显式表示改进概率 $p(I|A^{\pi_{\text{ref}}}(\mathrm{o}, \mathrm{a}))$，前提是我们训练策略，使其能够同时表示 $\pi_{\text{ref}}(\mathrm{a}|\mathrm{o}\ \ell)$ 以及

$\pi_{\text{ref}}(\mathbf{a}|I, \mathbf{o}, \ell)$. This principle is similar to the approach in classifier-free guidance (CFG), where a diffusion model is trained to model the data both with and without a conditioning variable [4]. We assume the improvement indicator I follows a delta distribution

$$p(I|A^{\pi_{\text{ref}}}(o, a, \ell)) = \delta(A^{\pi_{\text{ref}}}(o, a, \ell) > \epsilon_\ell),$$

with a task dependent improvement threshold ϵ_ℓ. This threshold allows us to control the optimality indicator, and minimizes the need for finding an attenuation factor β to sharpen the improvement conditioned distribution after training.[1] The policy objective then corresponds to minimizing the following negative log-likelihood:

$$\min_\theta \mathbb{E}_{\mathcal{D}_{\pi_{\text{ref}}}}\Big[-\log \pi_\theta(\mathbf{a}_t|\mathbf{o}_t, \ell) - \alpha \log \pi_\theta(\mathbf{a}_t|I_t, \mathbf{o}_t, \ell)\Big], \quad \text{where } I_t = \mathbb{1}\big(A^{\pi_{\text{ref}}}(\mathbf{o}_t, \mathbf{a}_t, \ell) > \epsilon_\ell\big). \tag{3}$$

The advantage values $A^{\pi_{\text{ref}}}(\mathbf{o}_t, \mathbf{a}_t, \ell)$ are obtained from the value function in the previous section, and α is a trade-off hyperparameter.

In practice, $\mathcal{D}_{\pi_{\text{ref}}}$ contains data from a mixture of sources, including demonstrations and autonomous task attempts collected by previously deployed policies. The policy extraction objective applies to all such data once each action has been assigned an indicator I_t from its estimated advantage. Section V-C describes how these indicators are assigned in our full training pipeline, including the treatment of teleoperated interventions. As we will discuss in Section V, our VLA model produces both discrete and continuous outputs, with the continuous distribution represented via flow matching. Therefore, the real training objective combines likelihoods for the discrete values with the flow matching objective for the continuous values.

V. Implementation, Model, and System Details

We instantiate RECAP with a VLA model, based on the $\pi_{0.6}$ VLA [76], which improves upon $\pi_{0.5}$ [55] by leveraging a larger backbone and more diverse conditioning. As shown in Figure 4, $\pi^*_{0.6}$ modifies $\pi_{0.6}$ by introducing conditioning on the binarized advantage indicator I_t, making it suitable for RL. To estimate advantage during VLA training, we also train a VLM-initialized value function. In this section, we first explain how we extend $\pi_{0.6}$ to incorporate advantages, then describe the reward and value function, and then elaborate on our training and data collection processes.

A. From $\pi_{0.6}$ to $\pi^*_{0.6}$ with advantage conditioning

$\pi^*_{0.6}$ uses the same base VLA architecture as $\pi_{0.6}$, but adds advantage conditioning for RL by including a binarized improvement indicator as an additional text input. Concretely, we input "Advantage: positive" when $I_t = \text{True}$ and "Advantage: negative" otherwise. The advantage indicator appears in the training sequence after $\hat{\ell}$ but before the discretized and continuous actions, such that only the action log-likelihoods are affected. Apart from this additional conditioning input, the VLA model is the same as detailed in the $\pi_{0.6}$ model card [76]. Following $\pi_{0.5}$ we train the continuous actions using the flow matching loss [77] and the discrete actions with autoregressive cross-entropy loss. It is possible to draw a close parallel between flow matching and diffusion (under some assumptions), and the latter in turn can be interpreted as a lower bound on the log-likelihood [78], so we can roughly motivate the sum of the log-likelihood of the discrete actions and the flow matching loss on the continuous actions as a lower bound on the overall action likelihood:

$$\log \pi_\theta(\mathbf{a}_{t:t+H}, a^\ell_{t:t+H}|I_t, \mathbf{o}_t, \ell, \hat{\ell}) \geq \mathbb{E}_{\eta,\omega}\Big[\log p_\theta(a^\ell_{t:t+H}|I_t, \mathbf{o}_t, \ell, \hat{\ell}) - \alpha_\eta \left\|\omega - \mathbf{a}_{t:t+H} - f_\theta(\mathbf{a}^{\eta,\omega}_{t:t+H}, I_t, \mathbf{o}_t, \ell, \hat{\ell})\right\|^2\Big], \tag{4}$$

with $\mathbf{a}^{\eta,\omega}_{t:t+H} = \eta\mathbf{a}_{t:t+H} + (1-\eta)\omega$, $\omega \sim \mathcal{N}(0, \mathbf{I})$ denoting the noised action, where $\eta \in [0, 1]$ is the flow matching time index and f_θ denotes the continuous outputs of the diffusion expert. α_η is a loss weighting term. Full details for the loss are provided in Appendix C. During training, we randomly omit the indicator I_t instead of tuning the loss multiplier α to allow us to either directly sample from the policy with $I_t = \text{True}$ (which corresponds to setting $\beta = 1$ in Equation (2)), or to use both a conditional and unconditional model to implement classifier-free guidance (CFG). See Appendix E for details.

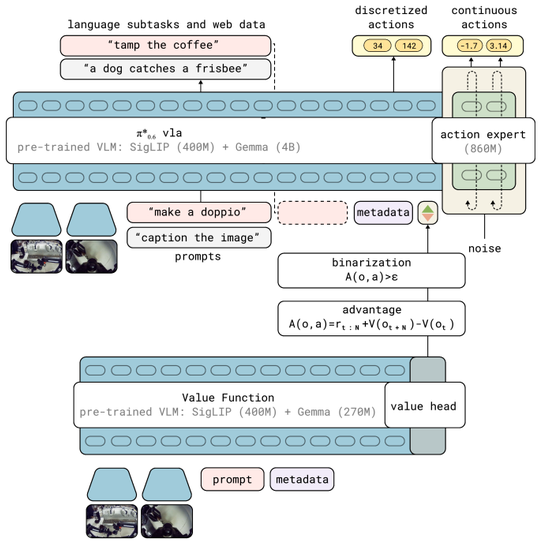


Fig. 4: **Interaction between $\pi^*_{0.6}$ and value function in RECAP.** The $\pi^*_{0.6}$ VLA uses a pre-trained VLM backbone. Training follows the KI recipe [75], with next-token prediction on many data sources in pre-training, and a flow-matching action-expert with stop gradient. Uniquely, the $\pi^*_{0.6}$ VLA is conditioned on a binarized advantage indicator, obtained from a separate value function initialized from a pre-trained but smaller VLM.

[1] Prior work [4] instead uniformly chose $\epsilon = 0$ and tuned β at test time, as in CFG. However, high CFG weights can drive the action distribution to the corners of its support (leading to aggressive behavior) and would not affect the autoregressive part of the model. We found it easier to obtain good results by instead using the threshold ϵ_ℓ to trade off regularization and optimality.

$\pi_{\text{ref}}(\mathrm{a}|I, \mathrm{o}\,\ell)$。这一原理类似于无分类器引导（CFG）中的方法，即训练扩散模型时，在有条件变量和没有条件变量[4]两种情况下都对数据建模。我们假设改进指示符 I 服从 delta 分布

$$p(I|A^{\pi_{\text{ref}}}(o, a, \ell)) = \delta(A^{\pi_{\text{ref}}}(o, a, \ell) > \epsilon_\ell),$$

采用与任务相关的改进阈值 ϵ_ℓ。该阈值使我们能够控制最优性指标，并最大程度减少在训练后寻找衰减因子 β 以锐化以改进为条件的分布的需求。[1] 随后，策略目标对应于最小化以下负对数似然：

$$\min_\theta \mathbb{E}_{\mathcal{D}_{\pi_{\text{ref}}}}\Big[-\log \pi_\theta(\mathbf{a}_t|\mathbf{o}_t, \ell) - \alpha \log \pi_\theta(\mathbf{a}_t|I_t, \mathbf{o}_t, \ell)\Big], \quad \text{where } I_t = \mathbb{1}\big(A^{\pi_{\text{ref}}}(\mathbf{o}_t, \mathbf{a}_t, \ell) > \epsilon_\ell\big). \tag{3}$$

优势值 $A^{\pi_{\text{ref}}}(\mathrm{o}_t, \mathrm{a}_t, \ell)$ 来自上一节中的价值函数，而 α 是一个权衡超参数。

在实践中，$\mathcal{D}_{\pi_{\text{ref}}}$ 包含来自多种来源的数据，包括演示以及由先前部署的策略收集的自主任务尝试。一旦根据每个动作的估计优势为其分配了指示符 I_t，策略提取目标就适用于所有此类数据。第 V-C 节描述了这些指示符在我们完整训练流程中如何分配，包括对遥操作干预的处理。正如我们将在第 V 节中讨论的，我们的 VLA 模型同时产生离散和连续输出，其中连续分布通过流匹配表示。因此，实际训练目标将离散值的似然与连续值的流匹配目标结合起来。

V. 实现、模型与系统细节

我们用一个VLA模型来实例化RECAP，该模型基于 $\pi_{0.6}$ VLA [76]，，并通过利用更大的骨干网络和更多样的条件信息改进了 $\pi_{0.5}$ [55]。如图4所示，$\pi^*_{0.6}$ 通过引入对二值化优势指标 I_t的条件化来修改 $\pi_{0.6}$，使其适用于强化学习。为了在VLA训练期间估计优势，我们还训练了一个由VLM初始化的价值函数。在本节中，我们首先说明如何扩展 $\pi_{0.6}$ 以纳入优势，然后描述奖励和价值函数，最后详述我们的训练与数据收集流程。

A. 从 $\pi_{0.6}$ 到 $\pi^*_{0.6}$ 的优势条件化

$\pi^*_{0.6}$ 使用与 $\pi_{0.6}$ 相同的基础 VLA 架构，但通过将二值化的改进指示符作为额外的文本输入，为强化学习加入优势条件。具体而言，当 I_t =True 时，我们输入“Advantage: positive”；否则输入“Advan- tage: negative”。优势指示符出现在

[1] 先前的工作 [4] 则统一选择 $\epsilon = 0$，并在测试时调节 β，如同在 CFG 中那样。然而，较高的CFG权重可能将动作分布推向其支撑集的角落（导致激进行为），并且不会影响模型的自回归部分。我们发现，转而使用阈值 ϵ_ℓ 来权衡正则性与最优性，更容易获得良好结果。

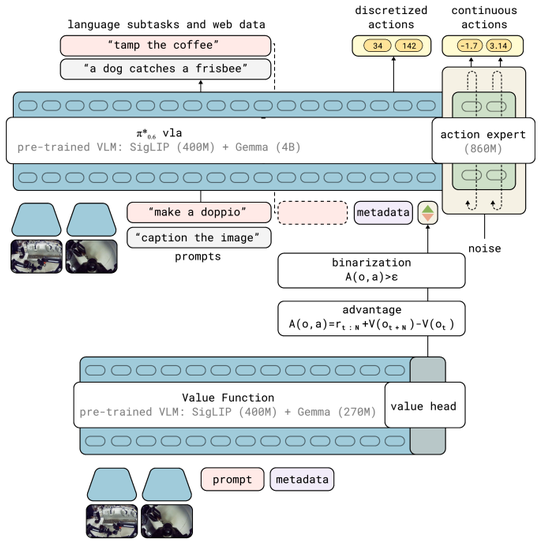


图 4：RECAP 中 $\pi^*_{0.6}$ 与价值函数之间的交互。$\pi^*_{0.6}$ VLA 使用预训练的 VLM 主干网络。训练遵循 KI 配方 [75]，：在预训练阶段对大量数据源采用下一词元预测，并采用带停止梯度的流匹配动作专家。独特之处在于，$\pi^*_{0.6}$ VLA 以二值化优势指标为条件，该指标来自一个单独的价值函数，而该价值函数由预训练但更小的 VLM 初始化。

训练序列中，位于 $\hat{\ell}$ 之后、离散动作和连续动作之前，因此只会影响动作对数似然。除这一额外的条件输入外，VLA 模型与 $\pi_{0.6}$ 模型卡 [76] 中详述的模型相同。遵循 $\pi_{0.5}$，我们使用流匹配损失 [77] 训练连续动作，并使用自回归交叉熵损失训练离散动作。可以在流匹配与扩散之间建立密切的类比（在某些假设下），而后者又可解释为对数似然 [78]，的下界，因此我们可以粗略地将离散动作的对数似然与连续动作上的流匹配损失之和，视为整体动作似然的下界：

$$\log \pi_\theta(\mathbf{a}_{t:t+H}, a^\ell_{t:t+H}|I_t, \mathbf{o}_t, \ell, \hat{\ell}) \geq \mathbb{E}_{\eta,\omega}\Big[\log p_\theta(a^\ell_{t:t+H}|I_t, \mathbf{o}_t, \ell, \hat{\ell}) - \alpha_\eta \left\|\omega - \mathbf{a}_{t:t+H} - f_\theta(\mathbf{a}^{\eta,\omega}_{t:t+H}, I_t, \mathbf{o}_t, \ell, \hat{\ell})\right\|^2\Big], \tag{4}$$

其中 $\mathrm{a}^{\eta\,\omega}_t$，$_{:t+H} = \eta \mathrm{a}_{t:t+H} + (1-\eta)\omega$，$\omega \sim \mathcal{N}(0, \mathrm{I})$ 表示加噪后的动作，其中 $\eta \in [0, 1]$ 是流匹配时间索引，f_θ 表示扩散专家的连续输出。α_η 是损失加权项。损失的完整细节见附录 C。在训练期间，我们随机省略指示符 I_t，而不是调整损失乘数 α，以便我们既可以直接在 I_t = 为 True 时从策略中采样（这对应于在式 (2) 中设置 $\beta = 1$），也可以同时使用条件模型和无条件模型来实现无分类器引导（CFG）。细节见附录 E。

B. Reward definition and value function training

Since our aim is to develop a general and broadly applicable method for improving the reliability and speed of VLAs from real-world experience, we use a generic reward definition based on episode-level success and time to completion. For successful episodes, we have a constant time penalty for every step (encouraging the robot to perform the task as quickly as possible). For failed episodes, we provide a large penalty C_{fail} (chosen so as to ensure that failed episodes have low returns) while successes end with reward 0. Formally, for an episode of length T, we define the reward for timestep t as:

$$r_t = \begin{cases} 0 & \text{if t = T and success} \\ -C_{\text{fail}} & \text{if t = T and failure} \\ -1 & \text{otherwise.} \end{cases} \quad (5)$$

Thus, the value function learns to predict the (negative of the) number of remaining steps until success for successful episodes, and a large negative value for failed episodes. In practice, we normalize the predicted values to be between $(-1, 0)$. Since we train on diverse tasks that have very different typical lengths, we normalize the values per task based on the maximum episode length of the task.

The value function takes as input the same language inputs as the $\pi^*_{0.6}$ VLA, and uses the same architecture design, with a smaller 670M parameter VLM backbone that is also initialized from Gemma 3 (see Figure 4). To prevent overfitting, we also co-train the value function on a small mixture of multi-modal web data. Figure 3 shows visualizations of the value function on some examples of successful and failure episodes, with additional visualizations in Figure 11 in Appendix B.

C. Pre-training and learning from experience

The data mixture used in the pre-training phase of our model largely follows the recipe used by $\pi_{0.5}$ [55], with vision-language data from the web, prediction of subtasks $\hat{\ell}$, and prediction of low-level actions on a variety of tasks from many different robots. During pre-training, we first train the value function on the dataset from Section V-B and compute the per-task improvement threshold ϵ_ℓ as the 30th percentile of predicted values for task ℓ. During VLA training, the value function is run on-the-fly to estimate $A^{\pi_{\text{ref}}}(\mathbf{o}_t, \mathbf{a}_t, \ell)$ for each example, which is then used to compute I_t based on ϵ_ℓ. As described in Section V-A, I_t is provided as input to $\pi^*_{0.6}$, yielding a single pre-trained model that represents $\pi_\theta(\mathbf{a}_t | I_t, \mathbf{o}_t, \ell)$ across the full pre-training mixture. Using a relatively small VLM backbone (670M) keeps on-the-fly value inference relatively inexpensive, adding approximately 5–10% wall-clock overhead during training.

For each target task, we then perform one or more iterations of RECAP independently. To start this policy improvement loop, we first warm-start the policy by fine-tuning $\pi^*_{0.6}$ with demonstration data $\mathcal{D}_\ell$ for the target task ℓ. In this one-time initialization stage, we fix the indicator I_t to True for all training examples, such that this stage corresponds to supervised fine-tuning (SFT). The goal of this stage is to obtain a sufficiently capable initial policy π^0_ℓ for collecting useful autonomous deployment data.

Thereafter, each policy improvement iteration consists of collecting additional deployment data, fine-tuning the value function on all data collected for the task so far, and then fine-tuning the policy with updated indicators I_t. During deployment data collection, we always collect some fully autonomous rollouts. We may additionally collect rollouts monitored by an expert teleoperator, who can intervene and provide demonstrations of corrective behavior. For autonomously collected data, $I_t \in \{\text{True}, \text{False}\}$ depends on the estimated advantage as described in Section IV-C. For interventions saved by the teleoperator, we set $I_t = \text{True}$, performing "teacher forcing" analogous to the standard DAgger treatment of interventions as expert behavior [5]. Although I_t could in principle be derived from the learned advantages for these segments as well, we deliberately assign positive labels to these corrections so that the policy treats them as positive-advantage examples during training. This choice has limitations: human operators cannot guarantee globally optimal behavior, and intervening during autonomous execution can be disruptive. Thus, corrections are useful for fixing large mistakes and overcoming challenges with exploration, while autonomous rollouts remain important for learning from the policy's own successes and failures. Generalizing this step is an interesting avenue for future work.

Both the value function and policy are fine-tuned from their respective pre-trained checkpoints, rather than their checkpoints from the last iteration. In practice, we found this to be useful for avoiding drift over multiple iterations. We can repeat this process for several iterations as needed, though in practice we found that even one iteration often leads to significantly improved results. The computational cost of each fine-tuning iteration varies with task and dataset size; in our experiments, each iteration averages roughly 200 H100 GPU-hours, with runs ranging from 20 to 520 H100 GPU-hours. At test time, RECAP introduces no additional inference overhead relative to the base VLA, since the deployed policy conditions directly on the positive advantage indicator rather than querying the value function online. On an H100 machine, end-to-end policy inference latency is approximately 100-150 ms.

VI. EXPERIMENTAL EVALUATION

Our experiments focus on the following questions:

(1) How much does RECAP improve the policy?

(2) How does RECAP improve $\pi_{0.6}$ over multiple iterations?

(3) How does the advantage-conditioned policy extraction method in RECAP compare to alternatives, such as AWR [67] or PPO-based diffusion training [21, 65]?

(4) Does using autonomous rollout data help compared to just using demonstrations and corrections?

(5) Can RECAP significantly alter policy behavior with relatively little data and remove a failure mode?

We use a two-arm robot platform shown in Figure 6 and consider a set of realistic tasks depicted in Figure 2: making espresso drinks, folding diverse laundry, and assembling boxes. Each task requires multiple steps, takes 5 to 15 minutes

B. 奖励定义与价值函数训练

由于我们的目标是开发一种通用且广泛适用的方法，以利用真实世界经验提升 VLA 的可靠性和速度，我们采用基于回合级成功与完成时间的通用奖励定义。对于成功的回合，我们对每一步施加恒定的时间惩罚（鼓励机器人尽可能快地执行任务）。对于失败的回合，我们提供较大的惩罚 C_{fail}（其选取以确保失败回合具有低回报），而成功回合以奖励 0 结束。形式上，对于长度为 T 的回合，我们将时间步 t 的奖励定义为：

$$r_t = \begin{cases} 0 & \text{if t = T and success} \\ -C_{\text{fail}} & \text{if t = T and failure} \\ -1 & \text{otherwise.} \end{cases} \quad (5)$$

因此，价值函数学习预测成功回合中直到成功所需的剩余步数（的负值），而对于失败回合则预测一个很大的负值。在实践中，我们将预测值归一化到 (−1,0) 之间。由于我们在典型长度差异很大的多样化任务上进行训练，我们会根据每个任务的最大回合长度对每个任务的值进行归一化。

价值函数以与 $\pi^*_{0.6}$ VLA相同的语言输入作为输入，并采用相同的架构设计，使用一个更小的 670M 参数 VLM 主干网络，该主干网络同样从 Gemma 3 初始化（见图 4）。为防止过拟合，我们还在一小部分多模态网络数据混合上对价值函数进行联合训练。图 3 展示了价值函数在成功和失败回合的一些示例上的可视化，更多可视化见图 11 附录 B。

C. 预训练与从经验中学习

我们模型预训练阶段所使用的数据混合，大体遵循 $\pi_{0.5}$ [55] 所用的配方，包含来自网络的视觉语言数据、子任务 $\hat{\ell}$ 的预测，以及对来自许多不同机器人的各种任务进行低层动作预测。在预训练期间，我们首先在第 V-B 节的数据集上训练价值函数，并计算每个任务的改进阈值 ϵ_ℓ，其取值为任务 ℓ 的预测值的第30百分位数。在 VLA 训练期间，价值函数会即时运行，以针对每个样本估计 $A^{\pi_{\text{ref}}}(\mathrm{o}_t, \mathrm{a}_t, \ell)$，该估计值随后用于基于 ϵ_ℓ 计算 I_t。如第 V-A 节所述，I_t 作为输入提供给 $\pi^*_{0.6}$，从而得到一个单一的预训练模型，该模型在整个预训练数据混合上表示 $\pi_\theta(\mathrm{a}_t | I_t, \mathrm{o}_t, \ell)$。使用相对较小的 VLM 主干（670M）可使即时价值推理的成本相对较低，在训练期间仅增加约 5–10% 的实际耗时开销。

对于每个目标任务，我们随后独立执行一次或多次 RECAP 迭代。为了启动这个策略改进循环，我们首先利用目标任务 ℓ 的示范数据 $\mathcal{D}_\ell$ 对策略进行微调 $\pi^*_{0.6}$，从而对策略进行热启动。在这个一次性初始化阶段，我们将所有训练样本的指示器 I_t 固定为 True，因此该阶段相当于监督微调（SFT）。该阶段的目标是获得一个具备足够能力的初始策略 π^0_ℓ，用于收集有用的自主部署数据。

此后，每次策略改进迭代都包括收集额外的部署数据、在迄今为止为该任务收集的所有数据上微调价值函数，然后用更新后的指标 I_t 微调策略。在部署数据收集期间，我们始终收集一些完全自主的轨迹采样。我们还可能收集由专家遥操作员监督的轨迹采样，专家可以介入并提供纠正行为的示范。对于自主收集的数据，$I_t \in \{\text{True,False}\}$ 取决于第 IV-C 节所述的估计优势。对于遥操作员保存的干预，我们设置 $I_t = \text{True}$，并执行“教师强制”，这类似于标准 DAgger 将干预视为专家行为的处理方式 [5]。尽管 I_t 原则上也可以从这些片段学到的优势中推导出来，但我们有意给这些纠正分配正标签，以便策略在训练期间将它们视为正优势示例。这一选择存在局限性：人类操作员无法保证全局最优行为，并且在自主执行期间进行干预可能会造成干扰。因此，纠正对于修复重大错误和克服探索中的挑战很有用，而自主轨迹采样对于从策略自身的成功和失败中学习仍然很重要。将这一步骤进行推广是未来工作的一个有趣方向。

价值函数和策略均从各自的预训练检查点进行微调，而不是从上一次迭代的检查点进行微调。在实践中，我们发现这有助于避免在多次迭代中发生漂移。我们可以按需重复此过程若干次迭代，不过在实验中我们发现，即使只进行一次迭代，也常常能显著提升结果。每次微调迭代的计算成本随任务和数据集规模而变化；在我们的实验中，每次迭代平均约需 200 H100 GPU 小时，各次运行的范围为 20 到 520 H100 GPU 小时。在测试时，RECAP 相对于基础 VLA 不会引入额外的推理开销，因为部署的策略直接以正优势指示符为条件，而不是在线查询价值函数。在 H100 机器上，端到端策略推理延迟约为 100-150 ms。

VI. 实验评估

我们的实验关注以下问题：（1）RECAP 能在多大程度上改进策略？（2）在多次迭代中，RECAP 如何改进 $\pi_{0.6}$？

(3) RECAP 中基于优势条件的策略提取方法与 AWR [67] 或基于 PPO 的扩散训练 [21, 65] 等替代方法相比如何？

(4) 与仅使用演示和纠正相比，使用自主 rollout 数据是否有帮助？

(5) RECAP 能否显著改变策略行为，例如在数据相对较少的情况下消除一种失效模式？

我们使用图 6 所示的双臂机器人平台，并考虑图 2 中描绘的一组真实任务：制作意式浓缩咖啡饮品、折叠各类衣物以及组装盒子。每个任务都需要多个步骤，耗时 5 到 15 分钟

Fig. 5: **Evaluation tasks and success criteria.** We evaluate three laundry variants, box assembly, and making espresso drinks. For each rollout, human annotators assess task completion using the task-specific quality criteria shown here, such as fold quality and placement for laundry or damage and stability of stacking for box assembly. These criteria are aggregated into a binary success label for reporting success rate.

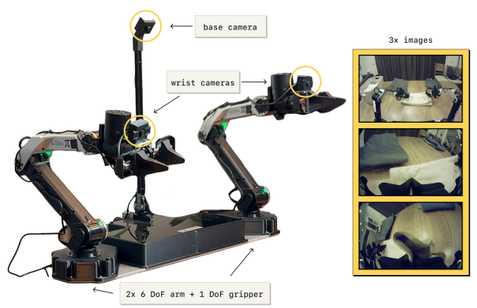


Fig. 6: **Our robot setup.** For experiments, we use a static bimanual system with two 6 DoF arms with parallel jaw grippers. The arms are controlled at 50 Hz with joint positions. Observations consist of joint and gripper positions, as well as images from three cameras: a base camera and wrist-mounted cameras.

to complete, and involves complex manipulation behaviors such as constrained forceful manipulation, pouring liquids, and manipulating cloth or cardboard. For each task, RECAP improves the policy using additional rollouts and interventions collected in the target deployment setting, and we evaluate the resulting policy in that same setting. We report *success rate* and *throughput*, where throughput is the expected number of successful task executions per hour. The evaluation tasks and success criteria are shown in Figure 5, with additional details in Appendix A.

A. Comparisons

We compare the following models:

- **Pre-trained** $\pi_{0.5}$ [55] is a baseline that represents a state-of-the-art pre-trained VLA that does not use RL.
- **Pre-trained** $\pi_{0.6}$ [76] is a baseline that uses the same architecture as $\pi^*_{0.6}$ but omits the advantage indicator I_t, and therefore does not use offline RL pre-training.
- **RL pre-trained** $\pi^*_{0.6}$ is our pre-trained model, pretrained with offline RL as described in Section V-C.
- $\pi^*_{0.6}$ **offline RL + SFT** is trained by fine-tuning the pre-trained $\pi^*_{0.6}$ model with hand-curated data. It represents the classic single-task fine-tuning recipe. During fine-tuning, the advantage values are fixed to True.
- $\pi^*_{0.6}$ **(Ours)** is our final model obtained by RECAP on the target task, fine-tuned on deployment data.
- $\pi^*_{0.6}$ **- Auton.** is an ablation model that is fine-tuned on the demonstration and intervention data only, removing autonomous deployment data.

We also consider two alternative policy extraction methods from the literature as baselines, both of which use the same on-robot data as RECAP but a different policy learning method:

- **AWR**. Starting from a pre-trained $\pi_{0.6}$ model (without advantage conditioning), we fine-tune wth advantage weighted regression [67] using the advantages extracted from our value function.
- **PPO**. We implement a variant of DPPO/FPO [21, 79] in which we calculate likelihoods based on the single step diffusion objective and use an alternative definition of the PPO constraint following SPO [80] (see Appendix D).

B. Quantitative results

(1) How much does RECAP *improve the policy?* We report throughput and success rates for each of the model variants on all tasks in Figures 7a and 7b. Across all tasks, the final $\pi^*_{0.6}$ model significantly improves over the base (supervised) $\pi_{0.6}$ model, the RL pre-trained $\pi^*_{0.6}$ model, and the **offline RL + SFT** $\pi^*_{0.6}$ model. Throughput more than doubles on the diverse laundry folding and espresso tasks from including on-robot data (the improvement from **offline RL + SFT** to the final $\pi^*_{0.6}$ model), and the rate of failure reduces by about a factor of two. On the T-shirts and shorts task, the success rate is already close to the maximum after the SFT phase, but throughput still increases by a significant margin.

On all tasks except diverse laundry, the success rate of the final $\pi^*_{0.6}$ model is in the 90%+ range, making it feasible to use in practice: making espresso drinks at an office or assembling boxes in a factory. For the box assembly task, Figure 7b (right) contains a breakdown of the task success over its four stages: picking up a box sheet, building the box, labeling the box,

图 5：评估任务与成功标准。我们评估三种洗衣变体、盒子组装以及制作浓缩咖啡饮品。对于每次 rollout，人类标注者使用此处展示的任务特定质量标准来评估任务完成情况，例如洗衣任务中的折叠质量和摆放位置，或盒子组装中的损坏情况和堆叠稳定性。这些标准会被汇总为一个二元成功标签，用于报告成功率。

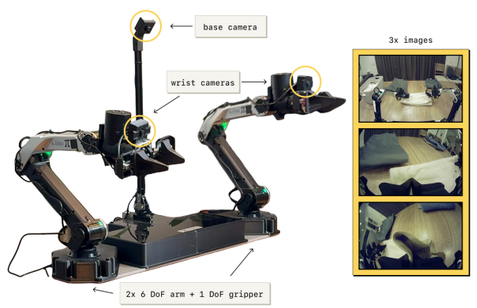


图 6：我们的机器人设置。在实验中，我们使用一个静态双臂系统，配有两个带平行夹爪的 6 自由度机械臂。机械臂通过关节位置以 50 Hz 进行控制。观测包括关节和夹爪位置，以及来自三个相机的图像：一个基座相机和腕部安装相机。

才能完成，并涉及受约束的强力操作、倾倒液体以及操作布料或纸板等复杂操作行为。对于每个任务，RECAP 利用在目标部署环境中收集的额外试运行和干预来改进策略，并且我们在同一环境中评估所得策略。我们报告成功率和吞吐量，其中吞吐量是每小时成功执行任务的期望数量。评估任务和成功标准如图 5 所示，更多细节见附录 A。

A. 比较

我们比较以下模型：

- 预训练的 $\pi_{0.5}$ [55] 是一个基线，代表一种不使用 RL 的最先进预训练 VLA。
- 预训练的 $\pi_{0.6}$ [76] 是一个基线，它使用与 $\pi^*_{0.6}$ 相同的架构，但省略了优势指示器 I_t，因此不使用离线 RL 预训练。
- RL 预训练的 $\pi^*_{0.6}$ 是我们的预训练模型，该模型按照第 V-C 节所述通过离线 RL 进行预训练。
- $\pi^*_{0.6}$ 离线 RL + SFT 通过使用人工整理的数据微调预训练 $\pi^*_{0.6}$ 模型来训练。它代表了

经典的单任务微调方案。在微调期间，优势值固定为 True。• $\pi^*_{0.6}$（我们的方法）是通过 RECAP 在目标任务上获得的最终模型，并在部署数据上进行了微调。• $\pi^*_{0.6}$ - Auton. 是一个消融模型，仅使用示范和干预数据进行微调，去除了自主部署数据。

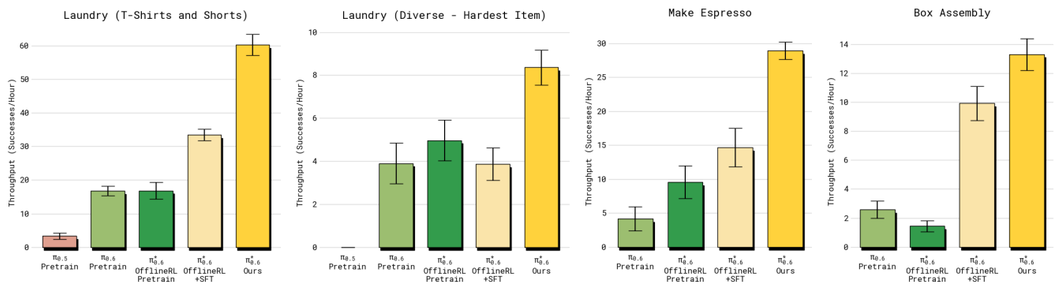
我们还考虑文献中的两种替代策略提取方法作为基线，二者都使用与 RECAP 相同的机器人数据，但采用不同的策略学习方法：

- AWR。从预训练的 $\pi_{0.6}$ 模型（不带优势条件）出发，我们使用从价值函数提取的优势，通过优势加权回归 [67] 进行微调。
- PPO。我们实现了 DPPO/FPO [21, 79] 的一种变体，其中我们基于单步扩散目标计算似然，并按照 SPO [80] 采用 PPO 约束的另一种定义（见附录 D ）。

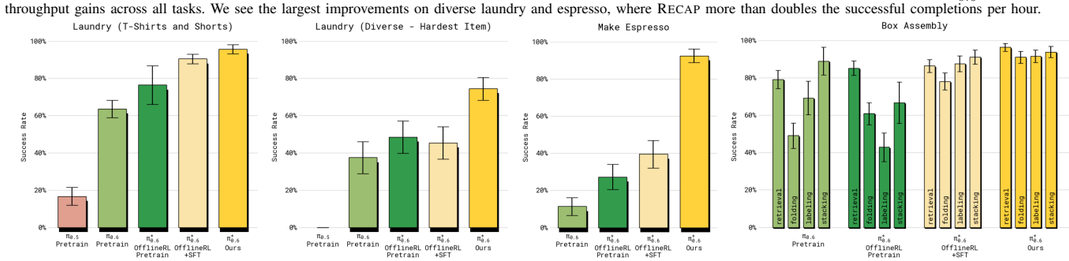
B. 定量结果

(1) RECAP 在多大程度上提升了策略？我们在图 7a 和 7b 中报告了所有任务上每个模型变体的吞吐量和成功率。在所有任务上，最终 $\pi^*_{0.6}$ 模型相较于基础（监督）$\pi_{0.6}$ 模型、RL 预训练的 $\pi^*_{0.6}$ 模型以及离线 RL + SFT $\pi^*_{0.6}$ 模型均有显著提升。在多样化的洗衣折叠和浓缩咖啡任务上，由于引入了机器人数据（即从离线 RL + SFT 到最终 $\pi^*_{0.6}$ 模型的提升），吞吐量提升了一倍以上，失败率降低了约一半。在 T 恤和短裤任务上，SFT 阶段后的成功率已经接近最大值，但吞吐量仍有显著提升。

除多样化衣物处理任务外，在其余所有任务上，最终 $\pi^*_{0.6}$ 模型的成功率均处于 90%+ 区间内，使其在实际应用中可行：在办公室制作浓缩咖啡饮品，或在工厂组装箱子。对于箱子组装任务，图 7b（右）给出了该任务在四个阶段的成功率细分：拿起箱板、组装箱子、给箱子贴标签，

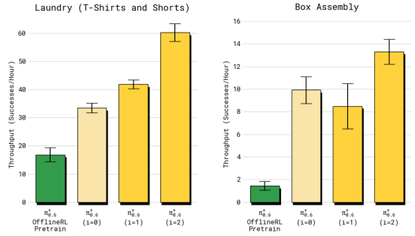


(a) **Throughput.** We report the number of successfully completed trials per hour for all tasks, with error bars denoting standard error. RECAP applied to $\pi^*_{0.6}$ yields substantial throughput gains across all tasks. We see the largest improvements on diverse laundry and espresso, where RECAP more than doubles the successful completions per hour.

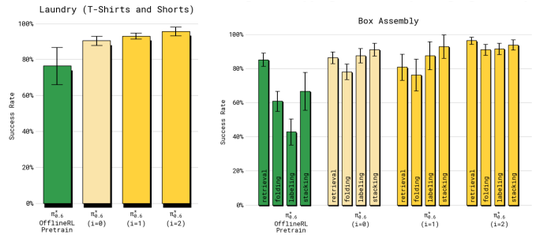


(b) **Success rates.** We report success rates with standard error for all tasks, finding that each component of RECAP consistently improves performance. The most challenging diverse laundry and espresso tasks exhibit the largest gains, with more than a $2\times$ reduction in failure rates. For box assembly, we further break down success rates per individual subtasks; RECAP achieves the most consistent and highest success across all subtasks.

Fig. 7: **Quantitative results** We show the efficacy of RECAP in improving policy performance with deployment data in terms of throughput and success rates.



(a) **Throughput.** Both tasks improve significantly in throughput as we take more iterations of RECAP, with box assembling first dropping and then improving significantly.



(b) **Success rate.** The laundry task quickly reaches the maximum success rate, while box assembly continues to improve.

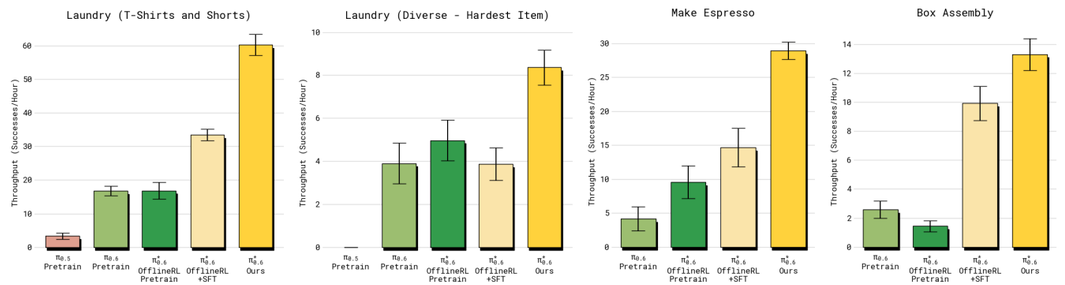
Fig. 8: **Policy improvement over multiple iterations.** RECAP can be used to iteratively improve policy performance.

and placing it at an available spot in a crate. $\pi^*_{0.6}$ attains higher success rates for all of the stages compared to the other models. The majority of failures on these stages happen because the policy runs out of time.

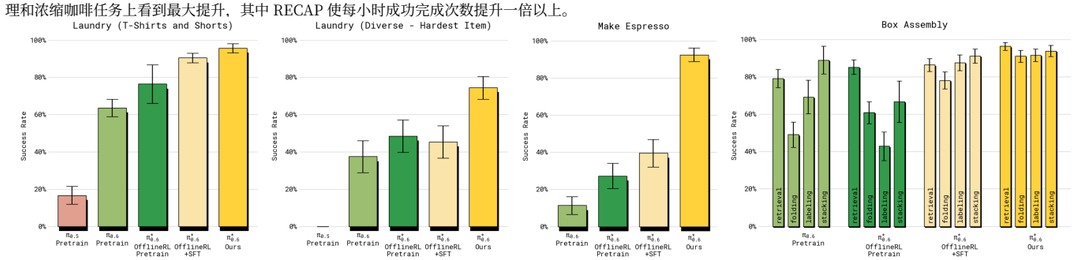
(2) How much does RECAP *improve $\pi^*_{0.6}$ over multiple iterations?* We next study how RECAP improves policies through multiple iterations. For the T-shirt folding task, we collect data using only autonomous evaluations (without human corrections) to perform policy improvement over two iterations, in order to evaluate how well our method can improve the policy completely autonomously. We collect 300 trajectories on four robots in each iteration. Box assembly uses both autonomous and teleoperated intervention data, with 600 autonomous trials and 360 trials with interventions per iteration.

We see the impact on throughput and success rate over iterations in Figure 8, comparing two iterations of RECAP, denoted by $i = 1$, $i = 2$ respectively. The final iteration, labeled (Ours), corresponds to the overall best result for these tasks presented in the previous section. We also compare the initial data collection policy, which uses the offline RL pre-trained $\pi^*_{0.6}$ model with SFT fine-tuning.

For both tasks, $\pi^*_{0.6}$ improves throughput over the two iterations. In the laundry task, we see steady improvement yielding an overall 50% improvement in throughput. Here, the first iteration already raises the success rate to over 90%, while the second iteration mainly improves throughput. For the box assembly task, we see clear improvements in the success rate over both iterations. While there are still some failures (especially when placing the box on the stack at the end), the final policy achieves a success rate of about 90% both for folding the box and labeling it in the allocated time limit of 600 seconds. Here we see a $2\times$ improvement in throughput during the second iteration.

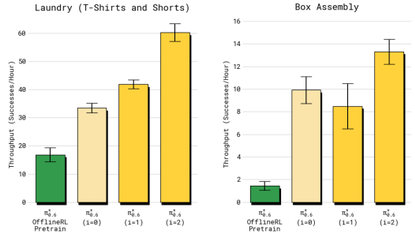


(a) 吞吐量。我们报告所有任务每小时成功完成的试验次数，误差条表示标准误。将 RECAP 应用于 $\pi^*_{0.6}$ 后，所有任务均获得显著的吞吐量提升。我们在多样化衣物处理和浓缩咖啡任务上看到最大提升，其中 RECAP 使每小时成功完成次数提升一倍以上。

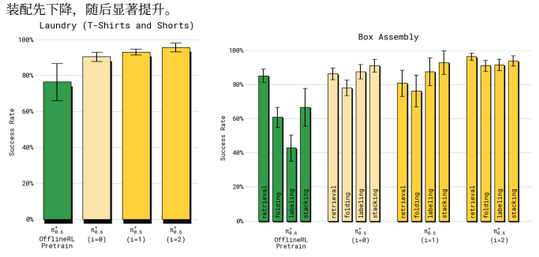


(b) 成功率。我们报告所有任务的成功率及标准误，发现 RECAP 的每个组件都能持续提升性能。最具挑战性的多样化衣物处理和浓缩咖啡任务提升最大，失败率降低超过 $2\times$ 。对于盒子装配，我们进一步按各个子任务细分成功率；RECAP 在所有子任务上取得了最稳定且最高的成功率。

图 7：定量结果 我们从吞吐量和成功率方面展示了 RECAP 利用部署数据提升策略性能的效果。



(a) 吞吐量。随着我们进行更多轮 RECAP 迭代，两项任务的吞吐量均显著提升，其中盒子装配先下降，随后显著提升。



(b) 成功率。洗衣任务很快达到最高成功率，而盒子组装任务则持续提升。

图 8：多次迭代中的策略改进。RECAP 可用于迭代提升策略性能。

并将其放置到板条箱中的空位。与其他模型相比， $\pi^*_{0.6}$ 在所有阶段都取得了更高的成功率。这些阶段的大多数失败是因为策略超时。

(2) RECAP 在多次迭代中能将 $\pi^*_{0.6}$ 提升多少？我们接下来研究 RECAP 如何通过多次迭代改进策略。对于 T 恤折叠任务，我们仅使用自主评估（不进行人工纠正）收集数据，以在两次迭代中进行策略改进，从而评估我们的方法能够在多大程度上完全自主地改进策略。每次迭代中，我们在四台机器人上收集 300 条轨迹。盒子组装同时使用自主数据和遥操作干预数据，每次迭代包括 600 次自主试验和 360 次带干预的试验。

我们在图 8 中可以看到多次迭代对吞吐量和成功率的影响，其中比较了 RECAP 的两次迭代，分别记为 $i = 1$、$i = 2$ 。最终迭代标记为（Ours），对应于上一节中给出的这些任务的整体最佳结果。我们还比较了初始数据收集策略，该策略使用经过 SFT 微调的离线 RL 预训练 $\pi^*_{0.6}$ 模型。

对于这两项任务， $\pi^*_{0.6}$ 在两次迭代中均提升了吞吐量。在洗衣任务中，我们看到了稳步提升，使吞吐量总体提高了 50%。在这里，第一次迭代就已将成功率提高到 90% 以上，而第二次迭代主要提升了吞吐量。对于盒子组装任务，我们看到成功率在两次迭代中都有明显提升。尽管仍有一些失败（尤其是在最后将盒子放到堆叠处时），但最终策略在 600 秒的分配时间限制内，无论是折叠盒子还是为其贴标签，都达到了约 90% 的成功率。在这里，我们看到第二次迭代期间吞吐量实现了 $2\times$ 的提升。

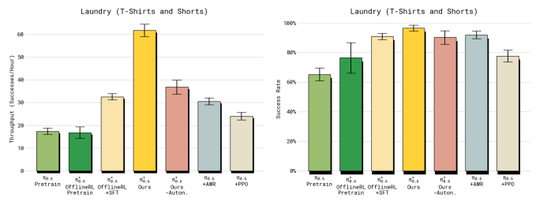


Fig. 9: **Comparison of different policy extraction methods.** RECAP achieves the highest throughput for the laundry task compared to alternative policy extraction methods like AWR and PPO. The full RECAP recipe (Ours) is better than training without autonomous data (Ours-Auton.).

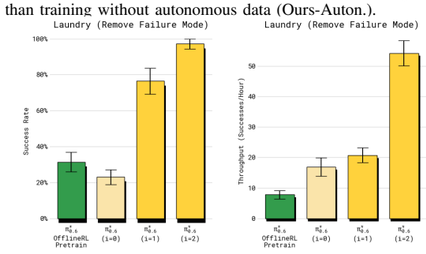


Fig. 10: **Failure mode removal.** RECAP is effective at removing failure modes that would be considered non successful under the strict criteria.

(3) How does the advantage-conditioned policy extraction method in RECAP *compare to other methods?* To answer this question, we use the T-shirts and Shorts task, and to ensure a controlled comparison, we use the same data that was used to train our final model for all methods. This provides a slight advantage to the baselines, since they have access to better data that was collected while running RECAP. As shown in Figure 9, while both AWR and PPO can attain reasonable results, they both fall far short of our method, and struggle to improve over the offline RL + SFT $\pi^*_{0.6}$ model. For PPO, we had to use a small trust-region constraint ($\eta = 0.01$) to stabilize training in this off-policy setting, and while this makes training stable, the method does not achieve good performance. AWR can achieve a reasonable success rate, but leads to much slower policies with lower throughput.

(4) Does using autonomous rollout data help compared to just using demonstrations and corrections? As shown in Figure 9 we find that RECAP without autonomous data (- Auton.) performs worse than using the full combination of demonstration, corrections and autonomous data for the laundry task (T-shirts and Shorts). We find that the relative usefulness of correction vs. autonomous data is task dependent but generally observe best results when using both.

(5) Can RECAP *significantly alter policy behavior with relatively little data and remove a failure mode?* While the preceding experiments have focused on holistic end-to-end evaluations of policy performance, here we zoom in on a specific failure mode to examine whether RL training with RECAP can remove a specific mistake from the policy. For this experiment, we use a version of the laundry task with a strict success criterion, which requires the policy to fold a T-shirt with the collar centered and facing up. Each episode is initialized with a specific adversarial condition in which the shirt is placed flat on the table in such a way that the baseline offline RL + SFT policy often fails to fold it correctly. As shown in Figure 10, applying RECAP in this setting for two iterations (collecting 600 trajectories in each iteration) results in a policy that succeeds 97% of the time, and with high speed. Thus we conclude that RECAP can be effective at removing specific failure modes, even when learning entirely from autonomously collected deployment data.

VII. CONCLUSION

Training policies that can achieve human-like robustness, speed, and fluency on real-world tasks presents a major challenge in robotic learning. This paper explores how learning from experience, through a combination of DAgger-style corrections and RL, can help address this challenge. We introduce RECAP, a scalable RL-based method for training VLAs that uses advantage conditioning for policy extraction, learning from autonomous trials, reward feedback, and targeted human interventions. We evaluate a model trained with RECAP, $\pi^*_{0.6}$, on realistic tasks including making espresso drinks, folding diverse laundry, and assembling boxes. Our experiments show that RECAP improves both success rate and throughput, more than doubling throughput on some of the harder tasks while reducing failures by approximately 2×.

There are several directions for improvement with RECAP. First, our system is not fully autonomous: it relies on human labeling and effort for reward feedback, interventions, and episode resets. Second, our system is relatively naïve in how it approaches exploration: we rely on the stochasticity of the policy and human interventions to explore new solutions. Third, several implementation choices in RECAP—including the value discretization, advantage threshold, and intervention labeling scheme—are practical design choices that worked well empirically, but may not be optimal; more systematic sensitivity studies would be valuable in future work. Lastly, RECAP performs iterated "offline" updates (i.e., it collects a batch of data, retrains the model, and repeats), rather than running a fully online RL loop where the model is updated while data is collected. More broadly, RL with VLAs presents a number of unique challenges, from the difficulty of large-scale RL training of high capacity models to sample complexity, autonomy, and delayed feedback. While existing RL frameworks designed for smaller-scale systems or LLMs can provide a good starting point, more research will be needed to make RL a practical tool for VLA training. We hope that our work represents a meaningful step in this direction.

REFERENCES

[1] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018. 1

[2] Sascha Lange, Thomas Gabel, and Martin A. Riedmiller. Batch reinforcement learning. In Marco A. Wiering and Martijn van Otterlo, editors, *Reinforcement Learning*, volume 12 of *Adaptation, Learning, and Optimization*, pages 45–73. Springer, 2012. doi: 10.1007/978-3-642-27645-3_2. 1

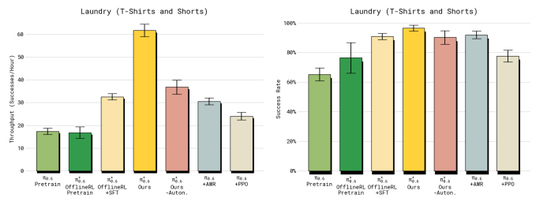


图 9：不同策略提取方法的比较。与 AWR 和 PPO 等替代策略提取方法相比，RECAP 在洗衣任务中实现了最高吞吐量。完整的 RECAP 方案（Ours）优于不使用自主数据训练（Ours-Auton.）。

图 10：失效模式移除。RECAP 能有效移除在严格标准下会被视为不成功的失效模式。

（3）RECAP 中基于优势条件的策略提取方法与其他方法相比如何？为了回答这个问题，我们使用 T 恤和短裤任务，并且为了确保受控比较，我们对所有方法都使用训练最终模型时所用的相同数据。这给基线方法带来了一定优势，因为它们可以使用在运行 RECAP 时收集到的更好的数据。如图 9 所示，尽管 AWR 和 PPO 都能取得合理的结果，但二者都远不及我们的方法，并且难以超越离线强化学习 + SFT $\pi^*_{0.6}$ 模型。对于 PPO，我们不得不使用一个较小的信赖域约束（$\eta = 0.01$），以在离策略设置中稳定训练；虽然这使训练稳定，但该方法并未取得良好性能。AWR 可以达到合理的成功率，但会导致策略慢得多且吞吐量更低。

(4) 与仅使用演示和纠正相比，使用自主 rollout 数据是否有帮助？如图 9 所示，我们发现，对于洗衣任务（T 恤和短裤），不使用自主数据的 RECAP（- Auton.）表现不如同时使用演示、纠正和自主数据的完整组合。我们发现纠正数据与自主数据的相对有用性取决于任务，但通常在使用两者时能获得最佳结果。

(5) RECAP 能否在数据相对较少的情况下显著改变策略行为，并消除某种失败模式？前述实验聚焦于策略性能的整体端到端评估，而这里我们聚焦于一个特定的失败模式，考察使用 RECAP 的强化学习训练能否消除策略中的某个特定错误。在本实验中，我们使用一个带有严格成功标准的叠衣任务版本，该标准要求策略折叠 T 恤时使领口居中且朝上。每个回合都初始化为一种特定的对抗性条件，其中衬衫被平放在桌子上，其放置方式会使基线离线 RL + SFT 策略经常无法正确折叠。如图 10 所示，在此设置下应用 RECAP 进行两次迭代（每次迭代收集 600 条轨迹），得到的策略成功率达到 97%，且速度很快。因此，我们得出结论：RECAP 能够有效消除特定的失败模式，即使完全从自主收集的部署数据中学习也是如此。

VII. 结论

训练能够在真实世界任务中达到类人鲁棒性、速度和流畅度的策略，是机器人学习中的一项重大挑战。本文探讨了如何通过结合 DAgger 式纠正和 RL 的经验学习来帮助应对这一挑战。我们提出 RECAP，这是一种可扩展的基于 RL 的方法，用于训练 VLA，它使用优势条件进行策略提取，并从自主试验、奖励反馈和有针对性的人工干预中学习。我们在包括制作浓缩咖啡饮品、折叠各类衣物和组装箱子等真实任务上评估了一个使用 RECAP 训练的模型 $\pi^*_{0.6}$。我们的实验表明，RECAP 提高了成功率和吞吐量，在某些更难的任务上将吞吐量提高了一倍以上，同时将失败减少约 2×。

RECAP 还有若干改进方向。首先，我们的系统并非完全自主：它依赖人工标注和人工投入来提供奖励反馈、进行干预以及重置回合。其次，我们的系统在探索方法上相对天真（naïve）：我们依赖策略的随机性和人工干预来探索新的解决方案。第三，RECAP 中的若干实现选择——包括价值离散化、优势阈值和干预标注方案——是经实践证明效果良好的实用设计选择，但可能并非最优；未来工作中开展更系统的敏感性研究将很有价值。最后，RECAP 执行的是迭代式“离线”更新（即收集一批数据、重新训练模型并重复此过程），而不是运行完全在线的 RL 循环——在后者中模型会在数据收集的同时进行更新。更广泛地说，将 RL 与 VLA 结合会带来许多独特挑战，从高容量模型大规模 RL 训练的难度，到样本复杂度、自主性和延迟反馈等问题。尽管为小规模系统或 LLM 设计的现有 RL 框架可以提供良好的起点，但要让 RL 成为 VLA 训练的实用工具，仍需开展更多研究。我们希望我们的工作是在这一方向上迈出的有意义的一步。

参考文献

[1] Richard S Sutton 与 Andrew G Barto。强化学习：导论。MIT 出版社，2018。1[2]Sascha Lange、Thomas Gabel 与 Martin A. Riedmiller。批量强化学习。载于 Marco A. Wiering 与 Martijn van Otterlo 编，《强化学习》，《适应、学习与优化》第 12 卷，第 45–73 页。Springer，2012。doi: 10.1007/978-3-642-27645-3\ 2。_1

[3] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020. 1

[4] Kevin Frans, Seohong Park, Pieter Abbeel, and Sergey Levine. Diffusion guidance is a controllable policy improvement operator. *arXiv preprint*, arXiv:2505.23458, 2025. 1, 3, 4, 5, 15

[5] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, pages 627–635, 2011. 2, 6

[6] Michael Laskey, Jonathan Lee, Roy Fox, Anca Dragan, and Ken Goldberg. Shiv: Reducing supervisor burden in dagger using support vectors for efficient learning from demonstrations in high dimensional state spaces. In *Proceedings of the 2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 462–469, 2016. doi: 10.1109/ICRA.2016.7487175. 2

[7] Michael Laskey, Jonathan Lee, Roy Fox, Anca D. Dragan, and Ken Goldberg. Dart: Noise injection for robust imitation learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1989–1998. PMLR, 2017.

[8] Eric Jang, Alex Irpan, Mohi Khansari, Daniel Kappler, Frederik Ebert, Corey Lynch, Sergey Levine, and Chelsea Finn. Bc-z: Zero-shot task generalization with robotic imitation learning. In *Conference on Robot Learning*, pages 991–1002. PMLR, 2022. 2

[9] Zheyuan Hu, Robyn Wu, Naveen Enock, Jasmine Li, Riya Kadakia, Zackory Erickson, and Aviral Kumar. Rac: Robot learning for long-horizon tasks by scaling recovery and correction. *arXiv preprint*, arXiv:2509.07953, 2025. 2

[10] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J Kochenderfer. Hg-dagger: Interactive imitation learning with human experts. In *ICRA*, 2019. 2

[11] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research*, 17(1):1334–1373, 2016. 2

[12] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, et al. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation. *arXiv preprint arXiv:1806.10293*, 2018.

[13] Ajay Mandlekar, Fabio Ramos, Byron Boots, Li Fei-Fei, Animesh Garg, and Dieter Fox. Iris: Implicit reinforcement without interaction at scale for learning control from offline robot manipulation data. *ICRA*, 2020.

[14] Archit Sharma, M. Ahmed Ahmed Rehaan Ahmad, and Chelsea Finn. Self-improving robots: End-to-end autonomous visuomotor reinforcement learning. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, volume 229, pages 3292–3308. PMLR, 2023.

[15] Russell Mendonca, Shikhar Bahl, and Deepak Pathak. Alan: Autonomously exploring robotic agents in the real world. In *Proceedings of the 2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3044–3050, 2023. doi: 10.1109/ICRA48891.2023.10013321.

[16] Russell Mendonca, Emmanuel Panov, Bernadette Bucher, Jiuguang Wang, and Deepak Pathak. Continuously improving mobile manipulation with autonomous real-world rl. In *Proceedings of the 8th Conference on Robot Learning (CoRL)*, pages 5204–5219, 2024.

[17] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning, 2024.

[18] Lars Ankile, Zhenyu Jiang, Rocky Duan, Guanya Shi, Pieter Abbeel, and Anusha Nagabandi. Residual off-policy rl for finetuning behavior cloning policies. *arXiv preprint arXiv:2509.19301*, 2025.

[19] Thomas Lampe, Abbas Abdolmaleki, Sarah Bechtle, Sandy H. Huang, Jost Tobias Springenberg, Michael Bloesch, Oliver Groth, Roland Hafner, Tim Hertweck, Michael Neunert, Markus Wulfmeier, Jingwei Zhang, Francesco Nori, Nicolas Heess, and Martin Riedmiller. Mastering stacking of diverse shapes with large-scale iterative reinforcement learning on real robots. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7772–7779, 2024. doi: 10.1109/ICRA57147.2024.10610297. 2

[20] Perry Dong, Suvir Mirchandani, Dorsa Sadigh, and Chelsea Finn. What matters for batch online reinforcement learning in robotics? *arXiv preprint*, arXiv:2505.08078, 2025. 2

[21] Allen Z. Ren, Justin Lidard, Lars Lien Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion Policy Policy Optimization. In *Proceedings of the 2025 International Conference on Learning Representations (ICLR)*, 2025. 6, 7, 15

[22] Kun Lei, Huanyu Li, Dongjie Yu, Zhenyu Wei, Lingxiao Guo, Zhennan Jiang, Ziyu Wang, Shiyu Liang, and Huazhe Xu. Rl-100: Performant robotic manipulation with real-world reinforcement learning. *arXiv preprint*, arXiv:2510.14830, 2025. 2

[23] Dmitry Kalashnkov, Jake Varley, Yevgen Chebotar, Ben Swanson, Rico Jonschkowski, Chelsea Finn, Sergey Levine, and Karol Hausman. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. *arXiv*, 2021. 2

[24] Abhishek Gupta, Justin Yu, Tony Z. Zhao, Vikash Kumar, Aaron Rovinsky, Kelvin Xu, Thomas Devlin, and Sergey Levine. Reset-free reinforcement learning via multi-

[3] Sergey Levine、Aviral Kumar、George Tucker 与 Justin Fu。离线强化学习：教程、综述及开放问题展望。arXiv 预印本 arXiv:2005.01643，2020。1[4] Kevin Frans、Seohong Park、Pieter Abbeel 与 Sergey Levine。扩散引导是一种可控的策略改进算子。arXiv 预印本，arXiv:2505.23458，2025。1, 3, 4, 5, 15[5] St´e phane Ross、Geoffrey Gordon 与 Drew Bagnell。将模仿学习与结构化预测归约为无悔在线学习。载于 AISTATS，第 627–635 页，2011。2, 6[6] Michael Laskey、Jonathan Lee、Roy Fox、Anca Dragan 与 Ken Goldberg。Shiv：利用支持向量降低 dagger 中的监督者负担，以便在高维状态空间中高效地从示范中学习。载于 2016 年 IEEE 国际机器人与自动化会议（ICRA）论文集，第 462–469 页，2016。doi: 10.1109/ICRA.2016.7487175。2[7] Michael Laskey、Jonathan Lee、Roy Fox、Anca D. Dragan 与 Ken Goldberg。Dart：面向稳健模仿学习的噪声注入。载于第 34 届国际机器学习大会（ICML）论文集，第 70 卷《机器学习研究论文集》，第 1989–1998 页。PMLR，2017。[8] Eric Jang、Alex Irpan、Mohi Khansari、Daniel Kappler、Frederik Ebert、Corey Lynch、Sergey Levine 与 Chelsea Finn。Bc-z：利用机器人模仿学习实现零样本任务泛化。载于机器人学习会议，第 991–1002 页。PMLR，2022。2[9]Zheyuan Hu、Robyn Wu、Naveen Enock、Jasmine Li、Riya Kadakia、Zackory Erickson 与 Aviral Kumar。Rac：通过扩展恢复与纠正来实现长时程任务的机器人学习。arXiv 预印本，arXiv:2509.07953，2025。2[10]Michael Kelly、Chelsea Sidrane、Katherine Driggs-Campbell 与 Mykel J Kochenderfer。Hg-dagger：与人类专家进行交互式模仿学习。载于 ICRA，2019。2[11] Sergey Levine、Chelsea Finn、Trevor Darrell 与 Pieter Abbeel。深度视觉运动策略的端到端训练。《机器学习研究杂志》，17(1)：1334–1373，2016。2[12] Dmitry Kalashnikov、Alex Irpan、Peter Pastor、Julian Ibarz、Alexander Herzog、Eric Jang、Deirdre Quillen、Ethan Holly、Mrinal Kalakrishnan、Vincent Vanhoucke 等。QT-Opt：面向基于视觉的机器人操作的可扩展深度强化学习。arXiv 预印本 arXiv:1806.10293，2018。[13]Ajay Mandlekar、Fabio Ramos、Byron Boots、Li Fei-Fei、Animesh Garg 与 Dieter Fox。Iris：用于从离线机器人操作数据中学习控制的规模化无交互隐式强化学习。ICRA，2020。[14]Archit Sharma、M. Ahmed Ahmed Rehaan Ahmad 与 Chelsea Finn。自我改进的机器人：端到端自主视觉运动强化学习。见《第7届机器人学习会议（CoRL）论文集》，第229卷，第3292–3308页。PMLR，2023。[15] Russell Mendonca、Shikhar Bahl 和 Deepak Pathak。Alan：在真实世界中自主探索机器人智能体。见《2023年IEEE国际机器人与自动化会议（ICRA）论文集》，第3044–3050页，2023。doi: 10.1109/ICRA48891.2023. 10013321。[16] Russell Mendonca、Emmanuel Panov、Bernadette Bucher、Jiuguang Wang 和 Deepak Pathak。通过自主真实世界强化学习持续改进移动操作。见《第8届机器人学习会议（CoRL）论文集》，第5204–5219页，2024。[17] Jianlan Luo、Zheyuan Hu、Charles Xu、You Liang Tan、Jacob Berg、Archit Sharma、Stefan Schaal、Chelsea Finn、Abhishek Gupta 和 Sergey Levine。Serl：面向样本高效机器人强化学习的软件套件，2024。[18] Lars Ankile、Zhenyu Jiang、Rocky Duan、Guanya Shi、Pieter Abbeel 和 Anusha Nagabandi。用于微调行为克隆策略的残差离策略强化学习。arXiv预印本 arXiv:2509.19301，2025。[19]Thomas Lampe、Abbas Abdolmaleki、Sarah Bechtle、Sandy H. Huang、Jost Tobias Springenberg、Michael Bloesch、Oliver Groth、Roland Hafner、Tim Hertweck、Michael Neunert、Markus Wulfmeier、Jingwei Zhang、Francesco Nori、Nicolas Heess 和 Martin Riedmiller。在真实机器人上通过大规模迭代强化学习掌握多样形状的堆叠。见《2024年IEEE国际机器人与自动化会议（ICRA）》，第7772–7779页，2024。doi: 10.1109/ICRA57147.2024.10610297。2[20]Perry Dong、Suvir Mirchandani、Dorsa Sadigh 和 Chelsea Finn。机器人中的批量在线强化学习什么最重要？arXiv预印本，arXiv:2505.08078，2025。2[21] Allen Z. Ren、Justin Lidard、Lars Lien Ankile、Anthony Simeonov、Pulkit Agrawal、Anirudha Majumdar、Benjamin Burchfiel、Hongkai Dai 和 Max Simchowitz。扩散策略策略优化。见《2025年国际学习表征会议（ICLR）论文集》，2025。6, 7, 15[22] Kun Lei、Huanyu Li、Dongjie Yu、Zhenyu Wei、Lingxiao Guo、Zhennan Jiang、Ziyu Wang、Shiyu Liang 和 Huazhe Xu。Rl-100：借助真实世界强化学习实现高性能机器人操作。arXiv预印本，arXiv:2510.14830，2025。2[23] Dmitry Kalashnkov、Jake Varley、Yevgen Chebotar、Ben Swanson、Rico Jonschkowski、Chelsea Finn、Sergey Levine 和 Karol Hausman。Mt-opt：大规模连续多任务机器人强化学习。arXiv，2021。2[24] Abhishek Gupta、Justin Yu、Tony Z. Zhao、Vikash Kumar、Aaron Rovinsky、Kelvin Xu、Thomas Devlin 和 Sergey Levine。无重置强化学习，通过多

task learning: Learning dexterous manipulation behaviors without human intervention. In *Proceedings of the 2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6664–6671, 2021. 2

[25] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X Lee, Maria Bauza, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, et al. Robocat: A self-improving foundation agent for robotic manipulation. *arXiv preprint arXiv:2306.11706*, 2023. 2

[26] Aviral Kumar, Anikait Singh, Frederik Ebert, Mitsuhiko Nakamoto, Yanlai Yang, Chelsea Finn, and Sergey Levine. Pre-training for robots: Offline reinforcement learning enables learning new tasks from a handful of trials. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023. doi: 10.15607/RSS.2023.XIX.019.

[27] Jingyun Yang, Max Sobol Mark, Brandon Vu, Archit Sharma, Jeannette Bohg, and Chelsea Finn. Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning. In *Proceedings of the 2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024. doi: 10.1109/ICRA57147.2024.10610421. 2

[28] Shuhan Tan, Kairan Dou, Yue Zhao, and Philipp Krähenbühl. Interactive post-training for vision-language-action models. *arXiv preprint*, arXiv:2505.17016, 2025. 2

[29] Guanxing Lu, Wenkai Guo, Chubin Zhang, Yuheng Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning. *arXiv preprint*, arXiv:2505.18719, 2025. 2

[30] Jijia Liu, Feng Gao, Bingwen Wei, Xinlei Chen, Qingmin Liao, Yi Wu, Chao Yu, and Yu Wang. What can rl bring to vla generalization? an empirical study. *arXiv preprint*, arXiv:2505.19789, 2025.

[31] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, Hongzhi Zang, Quanlu Zhang, Zhaofei Yu, Guoliang Fan, Tiejun Huang, Yu Wang, and Chao Yu. $\pi_{\texttt{rl}}$: Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint*, arXiv:2510.25889, 2025.

[32] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. Simplevla-rl: Scaling vla training via reinforcement learning. *arXiv preprint*, arXiv:2509.09674, 2025. 2

[33] Yanjiang Guo, Jianke Zhang, Xiaoyu Chen, Xiang Ji, Yen-Jen Wang, Yucheng Hu, and Jianyu Chen. Improving vision-language-action model with online reinforcement learning. *arXiv preprint*, arXiv:2501.16664, 2025. 2

[34] Wenli Xiao, Haotian Lin, Andy Peng, Haoru Xue, Tairan He, Yuqi Xie, Fengyuan Hu, Jimmy Wu, Zhengyi Luo, Linxi "Jim" Fan, Guanya Shi, and Yuke Zhu. Self-improving vision-language-action models with data generation via residual rl, 2025. 2

[35] Yuhui Chen, Shuai Tian, Shugao Liu, Yingting Zhou, Haoran Li, and Dongbin Zhao. Conrft: A reinforced fine-tuning method for vla models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025. 2

[36] Max Sobol Mark, Tian Gao, Georgia Gabriela Sampaio, Mohan Kumar Srirama, Archit Sharma, Chelsea Finn, and Aviral Kumar. Policy-agnostic rl: Offline rl and online rl fine-tuning of any class and backbone. *arXiv preprint*, arXiv:2412.06685, 2024. 2

[37] Mitsuhiko Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning*, pages 4996–5013. PMLR, 2025.

[38] Yang Zhang, Chenwei Wang, Ouyang Lu, Yuan Zhao, Yunfei Ge, Zhenglong Sun, Xiu Li, Chi Zhang, Chenjia Bai, and Xuelong Li. Align-then-steer: Adapting the vision-language action models through unified latent guidance. *arXiv preprint arXiv:2509.02055*, 2025. 2

[39] Andrew Wagenmaker, Mitsuhiko Nakamoto, Yunchu Zhang, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. Steering your diffusion policy with latent space reinforcement learning. In *Proceedings of the 9th Conference on Robot Learning (CoRL)*, 2025. 2

[40] Charles Xu, Qiyang Li, Jianlan Luo, and Sergey Levine. Rldg: Robotic generalist policy distillation via reinforcement learning. *arXiv preprint arXiv:2412.09858*, 2024. 2

[41] Dongchi Huang, Zhirui Fang, Tianle Zhang, Yihang Li, Lin Zhao, and Chunhe Xia. Co-rft: Efficient fine-tuning of vision-language-action models through chunked offline reinforcement learning. *arXiv preprint*, arXiv:2508.02219, 2025. 2

[42] Zijian Zhang, Kaiyuan Zheng, Zhaorun Chen, Joel Jang, Yi Li, Siwei Han, Chaoqi Wang, Mingyu Ding, Dieter Fox, and Huaxiu Yao. Grape: Generalizing robot policy via preference alignment. *arXiv preprint*, arXiv:2411.19309, 2024. 2

[43] Shaopeng Zhai, Qi Zhang, Tianyi Zhang, Fuxian Huang, Haoran Zhang, Ming Zhou, Shengzhe Zhang, Litao Liu, Sixu Lin, and Jiangmiao Pang. A vision-language-action-critic model for robotic real-world reinforcement learning. *arXiv preprint*, arXiv:2509.15937, 2025. 2, 3

[44] Seyed Kamyar Ghasemipour, Ayzaan Wahid, Jonathan Tompson, Pannag Sanketi, and Igor Mordatch. Self-improving embodied foundation models. *arXiv preprint*, arXiv:2509.15155, 2025. 2

[45] Jürgen Schmidhuber. Reinforcement learning upside down: Don't predict rewards — just map them to actions. *arXiv preprint*, arXiv:1912.02875, 2019. 3, 4

[46] Aviral Kumar, Xue Bin Peng, and Sergey Levine. Reward-conditioned policies. *CoRR*, abs/1912.13465, 2019. 4

任务学习：无需人工干预地学习灵巧操作行为。见《2021年IEEE国际机器人与自动化会议（ICRA）论文集》，第6664–6671页，2021。2[25] Konstantinos Bousmalis、Giulia Vezzani、Dushyant Rao、Coline Devin、Alex X Lee、Maria Bauza、Todor Davchev、Yuxiang Zhou、Agrim Gupta、Akhil Raju 等。Robocat：面向机器人操作的自改进基础智能体。arXiv预印本 arXiv:2306.11706，2023。2[26] Aviral Kumar、Anikait Singh、Frederik Ebert、Mitsuhiko Nakamoto、Yanlai Yang、Chelsea Finn 和 Sergey Levine。机器人预训练：离线强化学习使机器人能从少量试验中学习新任务。见《机器人学：科学与系统（RSS）论文集》，2023。doi: 10.15607/RSS.2023.XIX.019。[27]Jingyun Yang、Max Sobol Mark、Brandon Vu、Archit Sharma、Jeannette Bohg 和 Chelsea Finn。让机器人微调变得简单：为自主真实世界强化学习预训练奖励与策略。见《2024年IEEE国际机器人与自动化会议（ICRA）论文集》，2024。doi: 10.1109/ICRA57147.2024.10610421。2[28]Shuhan Tan、Kairan Dou、Yue Zhao 和 Philipp Kr¨ahenb¨uhl。面向视觉-语言-动作模型的交互式后训练。arXiv预印本，arXiv:2505.17016，2025。2[29]Guanxing Lu、Wenkai Guo、Chubin Zhang、Yuheng Zhou、Haonan Jiang、Zifeng Gao、Yansong Tang 和 Ziwei Wang。Vla-rl：借助可扩展强化学习实现娴熟且通用的机器人操作。arXiv预印本，arXiv:2505.18719，2025。[30] Jijia Liu、Feng Gao、Bingwen Wei、Xinlei Chen、Qingmin Liao、Yi Wu、Chao Yu 和 Yu Wang。强化学习能为VLA泛化带来什么？一项实证研究。arXiv预印本，arXiv:2505.19789，2025。[31] Kang Chen、Zhihao Liu、Tonghe Zhang、Zhen Guo、Si Xu、Hao Lin、Hongzhi Zang、Quanlu Zhang、Zhaofei Yu、Guoliang Fan、Tiejun Huang、Yu Wang 和 Chao Yu。$\pi_{\texttt{rl}}$：面向基于流的视觉-语言-动作模型的在线强化学习微调。arXiv预印本，arXiv:2510.25889，2025。[32] Haozhan Li、Yuxin Zuo、Jiale Yu、Yuhao Zhang、Zhaohui Yang、Kaiyan Zhang、Xuekai Zhu、Yuchen Zhang、Tianxing Chen、Ganqu Cui、Dehui Wang、Dingxiang Luo、Yuchen Fan、Youbang Sun、Jia Zeng、Jiangmiao Pang、Shanghang Zhang、Yu Wang、Yao Mu、Bowen Zhou 和 Ning Ding。Simplevla-rl：通过强化学习扩展VLA训练。arXiv预印本，arXiv:2509.09674，2025。2[33] Yanjiang Guo、Jianke Zhang、Xiaoyu Chen、Xiang Ji、Yen-Jen Wang、Yucheng Hu 和 Jianyu Chen。利用在线强化学习改进视觉-语言-动作模型。arXiv预印本，arXiv:2501.16664，2025。2[34] Wenli Xiao、Haotian Lin、Andy Peng、Haoru Xue、Tairan He、Yuqi Xie、Fengyuan Hu、Jimmy Wu、Zhengyi Luo、Linxi "Jim" Fan、Guanya Shi 和 Yuke Zhu。自

通过残差强化学习进行数据生成以改进视觉-语言-动作模型，2025。2[35] Yuhui Chen、Shuai Tian、Shugao Liu、Yingting Zhou、Haoran Li 和 Dongbin Zhao。Conrft：一种通过一致性策略对 VLA 模型进行强化微调的方法。arXiv 预印本 arXiv:2502.05450，2025。2[36] Max Sobol Mark、Tian Gao、Georgia Gabriela Sampaio、Mohan Kumar Srirama、Archit Sharma、Chelsea Finn 和 Aviral Kumar。策略无关强化学习：对任意类别和骨干网络进行离线强化学习与在线强化学习微调。arXiv 预印本，arXiv:2412.06685，2024。2[37]Mitsuhiko Nakamoto、Oier Mees、Aviral Kumar 和 Sergey Levine。引导你的通用模型：通过价值引导改进机器人基础模型。载于《机器人学习会议》，第 4996–5013 页。PMLR，2025。[38] Yang Zhang、Chenwei Wang、Ouyang Lu、Yuan Zhao、Yunfei Ge、Zhenglong Sun、Xiu Li、Chi Zhang、Chenjia Bai 和 Xuelong Li。先对齐再引导：通过统一潜空间引导适配视觉-语言动作模型。arXiv 预印本 arXiv:2509.02055，2025。2[39]Andrew Wagenmaker、Mitsuhiko Nakamoto、Yunchu Zhang、Seohong Park、Waleed Yagoub、Anusha Naga- bandi、Abhishek Gupta 和 Sergey Levine。用潜空间强化学习引导你的扩散策略。载于第九届机器人学习会议（CoRL）论文集，2025。2[40] Charles Xu、Qiyang Li、Jianlan Luo 和 Sergey Levine。Rldg：通过强化学习蒸馏机器人通用策略。arXiv 预印本 arXiv:2412.09858，2024。2[41] Dongchi Huang、Zhirui Fang、Tianle Zhang、Yihang Li、Lin Zhao 和 Chunhe Xia。Co-rft：通过分块离线强化学习高效微调视觉-语言-动作模型。arXiv 预印本，arXiv:2508.02219，2025。2[42]Zijian Zhang、Kaiyuan Zheng、Zhaorun Chen、Joel Jang、Yi Li、Siwei Han、Chaoqi Wang、Mingyu Ding、Dieter Fox 和 Huaxiu Yao。Grape：通过偏好对齐泛化机器人策略。arXiv 预印本，arXiv:2411.19309，2024。2[43] Shaopeng Zhai、Qi Zhang、Tianyi Zhang、Fuxian Huang、Haoran Zhang、Ming Zhou、Shengzhe Zhang、Litao Liu、Sixu Lin 和 Jiangmiao Pang。一种用于机器人真实世界强化学习的视觉-语言-动作-评论家模型。arXiv 预印本，arXiv:2509.15937，2025。2, 3[44] Seyed Kamyar Ghasemipour、Ayzaan Wahid、Jonathan Tompson、Pannag Sanketi 和 Igor Mordatch。自我改进的具身基础模型。arXiv 预印本，arXiv:2509.15155，2025。2[45] J¨urgen Schmidhuber。颠倒的强化学习：不要预测奖励——只需将它们映射为动作。arXiv 预印本，arXiv:1912.02875，2019。3, 4[46]Aviral Kumar、Xue Bin Peng 和 Sergey Levine。奖励条件策略。CoRR，abs/1912.13465，2019

[47] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS) 34*, 2021.

[48] David Brandfonbrener, Alberto Bietti, Jacob Buckman, Romain Laroche, and Joan Bruna. When does return-conditioned supervised learning work for offline reinforcement learning? In *Advances in Neural Information Processing Systems (NeurIPS) 35*, 2022. 4

[49] Scott Emmons, Benjamin Eysenbach, Ilya Kostrikov, and Sergey Levine. Rvs: What is essential for offline rl via supervised learning? In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022.

[50] Hiroki Furuta, Yusuke Matsuo, and Shixiang Shane Gu. Generalized decision transformer for offline hindsight information matching. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022.

[51] Taku Yamagata, Ahmed Khalil, and Raúl Santos-Rodríguez. Q-learning decision transformer: Leveraging dynamic programming for conditional sequence modelling in offline rl. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 38989–39007. PMLR, 2023.

[52] Qinqing Zheng, Amy Zhang, and Aditya Grover. Online decision transformer. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pages 27042–27059. PMLR, 2022.

[53] Jakub Grudzien Kuba, Pieter Abbeel, and Sergey Levine. Advantage-conditioned diffusion: Offline rl via generalization. 2023.

[54] Yueh-Hua Wu, Xiaolong Wang, and Masashi Hamaya. Elastic decision transformer. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.5555/3666122.3666936. 3

[55] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y Galliker, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *9th Annual Conference on Robot Learning*, 2025. 3, 5, 6, 7

[56] Lin Shao, Toki Migimatsu, Qiang Zhang, Kaiyuan Yang, and Jeannette Bohg. Concept2robot: Learning manipulation concepts from instructions and human demonstrations. In *Proceedings of Robotics: Science & Systems (RSS)*, 2020. doi: 10.15607/RSS.2020.XVI.082. 3

[57] Annie S. Chen, Suraj Nair, and Chelsea Finn. Learning generalizable robotic reward functions from "in-the-wild" human videos. In *Proceedings of Robotics: Science & Systems (RSS) 2021*, 2021.

[58] Suraj Nair, Eric Mitchell, Kevin Chen, Brian Ichter, Silvio Savarese, and Chelsea Finn. Learning language-conditioned robot behavior from offline data and crowd-sourced annotation. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 1303–1315. PMLR, 2022.

[59] Sumedh A. Sontakke, Jesse Zhang, Sébastien M.R. Arnold, Karl Pertsch, Erdem Bıyık, Dorsa Sadigh, Chelsea Finn, and Laurent Itti. Roboclip: One demonstration is enough to learn robot policies. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.

[60] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, Brian Ichter, Ted Xiao, Peng Xu, Andy Zeng, Tingnan Zhang, Nicolas Heess, Dorsa Sadigh, Jie Tan, Yuval Tassa, and Fei Xia. Language to rewards for robotic skill synthesis. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pages 374–404. PMLR, 2023.

[61] Jiahui Zhang, Yusen Luo, Abrar Anwar, Sumedh Anand Sontakke, Joseph J. Lim, Jesse Thomason, Erdem Bıyık, and Jesse Zhang. Rewind: Language-guided rewards teach robot policies without new demonstrations. In *Proceedings of the 9th Conference on Robot Learning (CoRL)*, 2025.

[62] Minttu Alakuijala, Reginald McLean, Isaac Woungang, Nariman Farsad, Samuel Kaski, Pekka Marttinen, and Kai Yuan. Video-language critic: Transferable reward functions for language-conditioned robotics. *Transactions on Machine Learning Research*, 2025:1–22, 2025. 3

[63] Yecheng Jason Ma, William Liang, Vaidehi Som, Vikash Kumar, Amy Zhang, Osbert Bastani, and Dinesh Jayaraman. Liv: Language-image representations and rewards for robotic control. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. 3

[64] Yecheng Jason Ma, Joey Hejna, Chuyuan Fu, Dhruv Shah, Jacky Liang, Zhuo Xu, Sean Kirmani, Peng Xu, Danny Driess, Ted Xiao, Osbert Bastani, Dinesh Jayaraman, Wenhao Yu, Tingnan Zhang, Dorsa Sadigh, and Fei Xia. Vision language models are in-context value learners. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. 3

[65] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 3, 4, 6, 15

[66] Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation. In *International Conference on Learning Representations*,

[47] Lili Chen、Kevin Lu、Aravind Rajeswaran、Kimin Lee、Aditya Grover、Michael Laskin、Pieter Abbeel、Aravind Srinivas 和 Igor Mordatch。决策 Transformer：通过序列建模进行强化学习。载于《神经信息处理系统进展》（NeurIPS）34，2021。[48] David Brandfonbrener、Alberto Bietti、Jacob Buckman、Romain Laroche 和 Joan Bruna。回报条件监督学习何时适用于离线强化学习？载于《神经信息处理系统进展》（NeurIPS）35，2022。4[49] Scott Emmons、Benjamin Eysenbach、Ilya Kostrikov 和 Sergey Levine。Rvs：通过监督学习进行离线强化学习的关键是什么？载于第十届国际学习表征会议（ICLR）论文集，2022。[50] Hiroki Furuta、Yusuke Matsuo 和 Shixiang Shane Gu。用于离线事后信息匹配的广义决策 Transformer。载于第十届国际学习表征会议（ICLR）论文集，2022。[51] Taku Yamagata、Ahmed Khalil 和 Ra´ul Santos- Rodr´ıguez。Q 学习决策 Transformer：利用动态规划进行离线强化学习中的条件序列建模。载于第四十届国际机器学习会议（ICML）论文集，《机器学习研究论文集》第 202 卷，第 38989–39007 页。PMLR，2023。[52] Qinqing Zheng、A my Zhang 和 Aditya Grover。在线决策 Transfor mer。载于第三十九届国际机器学习会议（ICML）论文集，《机器学习研究论文集》第 162 卷，第 27042–27059 页。PMLR，2022。[53] Jakub Grudzien Kuba、Pieter Abbeel 和 Sergey Levine。优势条件扩散：通过泛化进行离线强化学习。2023。[54] Yueh-Hua Wu、Xiaolong Wang 和 Masashi Hamaya。弹性决策 Transformer。载于第三十七届神经信息处理系统会议（NeurIPS）论文集，2023。doi: 10.5555/3666122.3666936。3[55]Kevin Black、Noah Brown、James Darpinian、Karan Dhabalia、Danny Driess、Adnan Esmail、Michael Robert Equi、Chelsea Finn、Niccolo Fusai、Manuel Y Galliker 等。 $\pi_{0.5}$：一种具有开放世界泛化能力的视觉-语言-动作模型。载于第九届机器人学习年度会议，2025。3, 5, 6, 7[56] Lin Shao、Toki Migimatsu、Qiang Zhang、Kaiyuan Yang 和 Jeannette Bohg。Concept2robot：从指令和人类演示中学习操作概念。载于《机器人学：科学与系统》（RSS）论文集，2020。doi：10.15607/RSS.2020.XVI.082。3[57] Annie S. Chen、Suraj Nair 和 Chelsea Finn。从“真实场景”人类视频中学习可泛化的机器人奖励函数。载于《机器人学：科学与系统》（RSS）2021 论文集，2021。[58]Suraj Nair、Eric Mitchell、Kevin Chen、Brian Ichter、Silvio Savarese 和 Chelsea Finn。从离线数据和众包标注中学习语言条件化的机器人行为。载于第5届机器人学习会议（CoRL）论文集，《机器学习研究会议录》第 164卷，第1303–1315页。PMLR，2022。[59]Sumedh A. Sontakke、Jesse Zhang、S´ebastien M.R. Arnold、Karl Pertsch、Erdem Bıyık、Dorsa Sadigh、Chelsea Finn 和 Laurent Itti。Roboclip：一次演示足以学习机器人策略。载于第37届神经信息处理系统会议（NeurIPS）论文集，2023。[60]Wenhao Yu、Nimrod Gileadi、Chuyuan Fu、Sean Kirmani、Kuang-Huei Lee、Montse Gonzalez Arenas、Hao-Tien Lewis Chiang、Tom Erez、Leonard Hasenclever、Jan Humplik、Brian Ichter、Ted Xiao、Peng Xu、Andy Zeng、Tingnan Zhang、Nicolas Heess、Dorsa Sadigh、Jie Tan、Yuval Tassa 和 Fei Xia。从语言到奖励：面向机器人技能合成。载于第7届机器人学习会议（CoRL）论文集，《机器学习研究会议录》第229卷，第374–404页。PMLR，2023。[61] Jiahui Zhang、Yusen Luo、Abrar Anwar、Sumedh Anand Sontakke、Joseph J. Lim、Jesse Thomason、Erdem Bıyık 和 Jesse Zhang。Rewind：语言引导的奖励无需新演示即可教会机器人策略。载于第9届机器人学习会议（CoRL）论文集，2025。[62] Minttu Alakuijala、Reginald McLean、Isaac Woungang、Nariman Farsad、Samuel Kaski、Pekka Marttinen 和 Kai Yuan。视频-语言评论器：面向语言条件化机器人的可迁移奖励函数。《机器学习研究汇刊》，2025：1–22，2025。3[63] Yecheng Jason Ma、William Liang、Vaidehi Som、Vikash Kumar、Amy Zhang、Osbert Bastani 和 Dinesh Jayaraman。Liv：面向机器人控制的语言-图像表示与奖励。载于第40届国际机器学习大会（ICML）论文集，2023。3[64] Yecheng Jason Ma、Joey Hejna、Chuyuan Fu、Dhruv Shah、Jacky Liang、Zhuo Xu、Sean Kirmani、Peng Xu、Danny Driess、Ted Xiao、Osbert Bastani、Dinesh Jayaraman、Wenhao Yu、Tingnan Zhang、Dorsa Sadigh 和 Fei Xia。视觉语言模型是上下文价值学习器。载于第13届国际学习表征会议（ICLR）论文集，2025。3[65] John Schulman、Filip Wolski、Prafulla Dhariwal、Alec Radford 和 Oleg Klimov。近端策略优化算法。arXiv 预印本 arXiv:1707.06347，2017。3, 4, 6, 15[66]Abbas Abdolmaleki、Jost Tobias Springenberg、Yuval Tassa、Remi Munos、Nicolas Heess 和 Martin Riedmiller。最大后验策略优化。载于国际学习表征会议。

2018. 3

[67] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019. 4, 6, 7

[68] Peter Dayan and Geoffrey E. Hinton. Using expectation-maximization for reinforcement learning. *Neural Computation*, 9(2):271–278, 1997. doi: 10.1162/neco.1997.9.2.271.

[69] Jan Peters, Katharina Mülling, and Yasemin Altün. Relative entropy policy search. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, AAAI'10, page 1607–1612. AAAI Press, 2010. 3

[70] Qing Wang, Jiechao Xiong, Lei Han, peng sun, Han Liu, and Tong Zhang. Exponentially weighted imitation learning for batched historical data. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, 2018. 3

[71] Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International conference on machine learning*, pages 449–458. PMLR, 2017. 4

[72] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *ICML*, 2018. 4

[73] Ziyu Wang, Alexander Novikov, Konrad Zolna, Josh S Merel, Jost Tobias Springenberg, Scott E Reed, Bobak Shahriari, Noah Siegel, Caglar Gulcehre, Nicolas Heess, and Nando de Freitas. Critic regularized regression. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 7768–7778, 2020. 4

[74] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022. 4

[75] Danny Driess, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z Ren, Homer Walke, Quan Vuong, Lucy Xiaoyang Shi, et al. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. In *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2025. 5

[76] Physical Intelligence Team. $\pi_{0.6}$ model card. Physical Intelligence model card PDF, 2025. 5, 7

[77] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. 5, 14

[78] Diederik Kingma and Ruiqi Gao. Understanding diffusion objectives as the elbo with simple data augmentation. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 65484–65516, 2023. 5, 15

[79] David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients, 2025. 7, 14, 15

[80] Zhengpeng Xie, Qiang Zhang, Fan Yang, Marco Hutter, and Renjing Xu. Simple policy optimization. In *Forty-second International Conference on Machine Learning (ICML)*, 2025. 7, 15

[81] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. 14

[67] Xue Bin Peng、Aviral Kumar、Grace Zhang 和 Sergey Levine。优势加权回归：简单且可扩展的离策略强化学习。arXiv 预印本 arXiv:1910.00177，2019。4, 6, 7[68] Peter Dayan 和 Geoffrey E. Hinton。使用期望最大化进行强化学习。《神经计算》，9(2)：271–278，1997。doi: 10.1162/neco.1997.9.2.271。[69] Jan Peters、Katharina M¨ulling 和 Yasemin Alt¨un。相对熵策略搜索。载于第24届AAAI人工智能会议（AAAI'10）论文集，第1607–1612页。AAAI Press，2010。3[70]Qing Wang、Jiechao Xiong、Lei Han、peng sun、Han Liu 和 Tong Zhang。面向批量历史数据的指数加权模仿学习。载于 S. Bengio、H. Wallach、H. Larochelle、K. Grauman、N. Cesa-Bianchi 和 R. Garnett 编，《神经信息处理系统进展》，第 31, 2018 3 卷。Marc G [71] Bellemare、Will Dabney 和 R´emi Munos。强化学习的分布视角。载于国际机器学习大会，第449–458页。PMLR，2017。4[72]Tuomas Haarnoja、Aurick Zhou、Pieter Abbeel 和 Sergey Levine。软演员-评论家：使用随机演员的离策略最大熵深度强化学习。ICML，2018。4[73] Ziyu Wang、Alexander Novikov、Konrad Zolna、Josh S Merel、Jost Tobias Springenberg、Scott E Reed、Bobak Shahriari、Noah Siegel、Caglar Gulcehre、Nicolas Heess 和 Nando de Freitas。评论家正则化回归。载于 H. Larochelle、M. Ranzato、R. Hadsell、M.F. Balcan 和 H. Lin 编，《神经信息处理系统进展》，第33卷，第7768–7778页，2020。4[74] Ilya Kostrikov、Ashvin Nair 和 Sergey Levine。基于隐式 Q 学习的离线强化学习。载于国际学习表征会议，2022。4[75] Danny Driess、Jost Tobias Springenberg、Brian Ichter、Lili Yu、Adrian Li-Bell、Karl Pertsch、Allen Z Ren、Homer Walke、Quan Vuong、Lucy Xiaoyang Shi 等。知识隔离的视觉-语言-动作模型：训练快、运行快、泛化更好。载于第37届神经信息处理系统会议（NeurIPS）论文集，2025。5[76] Physical Intelligence 团队。 $\pi_{0.6}$ 模型卡。Physical Intelligence 模型卡 PDF，2025。5, 7[77] Yaron Lip man、Ricky TQ Chen、Heli Ben-Hamu、Maximilian Nickel 和 Matt Le。用于生成建模的流匹配。arXiv 预印本 arXiv:2210.02747，2022。5, 14[78] Diederik Kingma 和 Ruiqi Gao。将扩散目标理解为带简单数据增强的 ELBO。载于 A. Oh、T. Naumann、A. Globerson、K. Saenko、M. Hardt 和 S. Levine 编，《神经信息处理系统进展》信息处理系统，第36卷，第65484–65516页，2023年。5, 15[79] David McAllister、Songwei Ge、Brent Yi、Chung Min Kim、Ethan Weber、Hongsuk Choi、Haiwen Feng 和 Angjoo Kanazawa。流匹配策略梯度，2025 年。7, 14, 15[80] Zhengpeng Xie、Qiang Zhang、Fan Yang、Marco Hutter 和 Renjing Xu。简单策略优化。载于第四十二届国际机器学习会议（ICML），2025年。7, 15[81] Kevin Black、Noah Brown、Danny Driess、Adnan Esmail、Michael Equi、Chelsea Finn、Niccolo Fusai、Lachy Groom、Karol Hausman、Brian Ichter、Szymon Jakubczak、Ti m Jones、Liyiming Ke、Sergey Levine、Adrian Li-Bell、Mohith Mothukuri、Suraj Nair、Karl Pertsch、Lucy Xiaoyang Shi、James Tanner、Quan Vuong、Anna Walling、Haohuan Wang 和 Ury Zhilinsky。 π_0：用于通用机器人控制的视觉-语言-动作流模型。arXiv 预印本 arXiv:2410.24164，2024年。14